

National Taiwan University

Tideology

Wayne Wei, Duda Liao, Ian Tsai

- 1 Contest
- 2 Mathematics
- 3 Data structures
- 4 Numerical
- 5 Number theory
- 6 Combinatorial
- 7 Graph
- 8 Geometry
- 9 Strings
- 10 Various

Contest (1)

todo.txt

4 lines

```
code run
chmod +x run
code tem.cpp
./run tem
```

template.cpp

45 lines

```
#include <bits/stdc++.h>
#include <bits/extc++.h>
using namespace std;
#define int long long
#define ll long long
#define all(x) x.begin(), x.end()
#define pb push_back
#define F first
#define S second
#define pii pair<int, int>
#define chmax(x, y) x = x > y ? x : y
#define chmin(x, y) x = x < y ? x : y
#define rep(i, a, b) for(int i = a; i < (b); ++i)
#define sz(x) (int)(x).size()
#define vi vector<int>
#ifdef tsao
template <typename T>
ostream& operator << (ostream &o, const vector<T> &v){
    o << "{"; int f = 0;
    for(auto &i : v) o << (f++ ? " " : "") << i;
    return o << "}";
}
void bug__(int c, auto ...a){
    cerr << "\e[1;" << c << "m";
    (... , (cerr << a << " "));
    cerr << "\e[0m\n";
}
#define bug_(c, x...) bug__(c, __LINE__, "[" + string(#x) + "]"
    , x)
#define bug(x...) bug_(32, x)
#define bugv(x...) bug_(36, vector(x))
```

2

3

4

6

10

12

13 fast.cpp

19

23

25

30 lines

```
#define safe bug_(33, "safe")
#else
#define bug(x...) void(0)
#define bugv(x...) void(0)
#define safe void(0)
#endif
void solve(){
}
signed main(){
    ios::sync_with_stdio(0);
    cin.tie(0);cout.tie(0);
    int t = 1;
    while(t--) solve();
}

#pragma GCC optimize("Ofast,no-stack-protector")
#pragma GCC optimize("no-math-errno,unroll-loops")
#pragma GCC target("sse,sse2,sse3,ssse3,sse4")
#pragma GCC target("popcnt,abm,mmx,avx,arch=skylake")
__builtin_ia32_ldmxcsr(__builtin_ia32_stmxcsr() | 0x8040)
#include<unistd.h>
char IB[1 << 16], OB[1 << 16]; int IP, IL, OP;
inline int RC(){
    if(IP == IL) IP = 0, IL = fread(IB, 1, sizeof IB, stdin);
    return IP == IL ? -1 : IB[IP++];
}
inline int R(){
    int c = RC(), s = 1; unsigned long long x = 0;
    while(c != -1 and c != '-' and (c < '0' or c > '9')) c = RC()
        ;
    if(c == '-') s = -1, c = RC();
    while(c >= '0' and c <= '9') x = x * 10 + (c ^ 48), c = RC();
    return s == 1 ? (int)x : (int)(0-x);
}
inline void PC(char c){
    if(OP == (int)sizeof OB) fwrite(OB, 1, OP, stdout), OP = 0;
    OB[OP++] = c;
}
inline void W(int n){
    char s[24]; int p = 0;
    unsigned long long x = n;
    if(n < 0) PC('-'), x = 0 - x;
    do s[p++] = char('0' + x % 10), x /= 10; while(x);
    while(p) PC(s[--p]);
}
struct Flush { ~Flush(){ fwrite(OB, 1, OP, stdout); } } flush;

run
1 lines
g++ -std=c++20 -Dtsao -Wall -Wextra -Wshadow -fsanitize=address
    -std=gnu++20 -O2 -pipe $1.cpp -o $1 && ./$1

.bashrc
3 lines
alias c='g++ -Wall -Wconversion -Wfatal-errors -g -std=c++17 \
    -fsanitize=undefined,address'
xmodmap -e 'clear lock' -e 'keycode 66=less greater' #caps = <

s.sh
12 lines
#!/usr/bin/env bash

i=0
while true
do
    echo $i
```

((++i))
./gen > in
./good < in > out1
./bad < in > out2
diff out1 out2 || break
done

.vimrc
6 lines
set cin aw ai is ts=4 sw=4 tm=50 nu noeb bg=dark ru cul
sy on | im jk <esc> | im kj <esc> | no ; :
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \ | tr -d '[:space:]' \
 \ | md5sum \ | cut -c-6

hash.sh
2 lines
code=\$(cpp -dD -P -fpreprocessed "\$@") || exit 1
printf '%s' "\$code" | tr -d '[:space:]' | md5sum | cut -c-6

troubleshoot.txt
52 lines
Pre-submit:
Write a few simple test cases if sample is not enough.
Are time limits close? If so, generate max cases.
Is the memory usage fine?
Could anything overflow?
Make sure to submit the right file.

Wrong answer:
Print your solution! Print debug output, as well.
Are you clearing all data structures between test cases?
Can your algorithm handle the whole range of input?
Read the full problem statement again.
Do you handle all corner cases correctly?
Have you understood the problem correctly?
Any uninitialized variables?
Any overflows?
Confusing N and M, i and j, etc.?
Are you sure your algorithm works?
What special cases have you not thought of?
Are you sure the STL functions you use work as you think?
Add some assertions, maybe resubmit.
Create some testcases to run your algorithm on.
Go through the algorithm for a simple case.
Go through this list again.
Explain your algorithm to a teammate.
Ask the teammate to look at your code.
Go for a small walk, e.g. to the toilet.
Is your output format correct? (including whitespace)
Rewrite your solution from the start or let a teammate do it.

Runtime error:
Have you tested all corner cases locally?
Any uninitialized variables?
Are you reading or writing outside the range of any vector?
Any assertions that might fail?
Any possible division by 0? (mod 0 for example)
Any possible infinite recursion?
Invalidated pointers or iterators?
Are you using too much memory?
Debug with resubmits (e.g. remapped signals, see Various).

Time limit exceeded:
Do you have any possible infinite loops?
What is the complexity of your algorithm?
Are you copying a lot of unnecessary data? (References)
How big is the input and output? (consider scanf)

Avoid vector, map. (use arrays/unordered_map)
What do your teammates think about your algorithm?

Memory limit exceeded:
What is the max amount of memory your algorithm should need?
Are you clearing all data structures between test cases?

Mathematics (2)

2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e & x &= \frac{ed - bf}{ad - bc} \\ cx + dy &= f & y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable x_i is given by

$$x_i = \frac{\det A'_i}{\det A}$$

where A'_i is A with the i 'th column replaced by b .

2.2 Recurrences

If $a_n = c_1a_{n-1} + \dots + c_ka_{n-k}$, and $r_1, \dots, r_k$ are distinct roots of $x^k - c_1x^{k-1} - \dots - c_k$, there are $d_1, \dots, d_k$ s.t.

$$a_n = d_1r_1^n + \dots + d_kr_k^n.$$

Non-distinct roots r become polynomial factors, e.g. $a_n = (d_1n + d_2)r^n$.

2.3 Trigonometry

$$\begin{aligned} \sin(v + w) &= \sin v \cos w + \cos v \sin w \\ \cos(v + w) &= \cos v \cos w - \sin v \sin w \end{aligned}$$

$$\begin{aligned} \tan(v + w) &= \frac{\tan v + \tan w}{1 - \tan v \tan w} \\ \sin v + \sin w &= 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2} \\ \cos v + \cos w &= 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2} \end{aligned}$$

$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where V, W are lengths of sides opposite angles v, w .

$$\begin{aligned} a \cos x + b \sin x &= r \cos(x - \phi) \\ a \sin x + b \cos x &= r \sin(x + \phi) \end{aligned}$$

where $r = \sqrt{a^2 + b^2}, \phi = \text{atan2}(b, a)$.

2.4 Geometry

2.4.1 Triangles

Side lengths: a, b, c

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles):
 $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc \left[1 - \left(\frac{a}{b + c} \right)^2 \right]}$$

Law of sines: $\frac{\sin \alpha}{a} = \frac{\sin \beta}{b} = \frac{\sin \gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc \cos \alpha$

Law of tangents: $\frac{a + b}{a - b} = \frac{\tan \frac{\alpha + \beta}{2}}{\tan \frac{\alpha - \beta}{2}}$

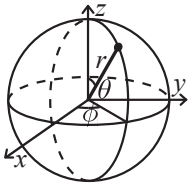
2.4.2 Quadrilaterals

With side lengths a, b, c, d , diagonals e, f , diagonals angle θ , area A and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin \theta = F \tan \theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180° ,
 $ef = ac + bd$, and $A = \sqrt{(p - a)(p - b)(p - c)(p - d)}$.

2.4.3 Spherical coordinates



$$\begin{aligned} x &= r \sin \theta \cos \phi & r &= \sqrt{x^2 + y^2 + z^2} \\ y &= r \sin \theta \sin \phi & \theta &= \text{acos}(z / \sqrt{x^2 + y^2 + z^2}) \\ z &= r \cos \theta & \phi &= \text{atan2}(y, x) \end{aligned}$$

2.5 Derivatives/Integrals

$$\begin{aligned} \frac{d}{dx} \arcsin x &= \frac{1}{\sqrt{1 - x^2}} & \frac{d}{dx} \arccos x &= -\frac{1}{\sqrt{1 - x^2}} \\ \frac{d}{dx} \tan x &= 1 + \tan^2 x & \frac{d}{dx} \arctan x &= \frac{1}{1 + x^2} \\ \int \tan ax &= -\frac{\ln |\cos ax|}{a} & \int x \sin ax &= \frac{\sin ax - ax \cos ax}{a^2} \\ \int e^{-x^2} &= \frac{\sqrt{\pi}}{2} \text{erf}(x) & \int x e^{ax} dx &= \frac{e^{ax}}{a^2} (ax - 1) \end{aligned}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

2.6 Sums

$$c^a + c^{a+1} + \dots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \dots + n = \frac{n(n + 1)}{2}$$

$$1^2 + 2^2 + 3^2 + \dots + n^2 = \frac{n(2n + 1)(n + 1)}{6}$$

$$1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^2(n + 1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n(n + 1)(2n + 1)(3n^2 + 3n - 1)}{30}$$

2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots, (-\infty < x < \infty)$$

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \dots, (-1 < x \leq 1)$$

$$\sqrt{1 + x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \dots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots, (-\infty < x < \infty)$$

2.8 Probability theory

Let X be a discrete random variable with probability $p_X(x)$ of assuming the value x . It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x x p_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where σ is the standard deviation. If X is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent X and Y ,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

2.8.1 Discrete distributions

Binomial distribution

The number of successes in n independent yes/no experiments, each which yields success with probability p is $\text{Bin}(n, p)$, $n = 1, 2, \dots$, $0 \leq p \leq 1$.

$$p(k) = \binom{n}{k} p^k (1 - p)^{n - k}$$

$$\mu = np, \sigma^2 = np(1 - p)$$

$\text{Bin}(n, p)$ is approximately $\text{Po}(np)$ for small p .

First success distribution

The number of trials needed to get the first success in independent yes/no experiments, each which yields success with probability p is $\text{Fs}(p)$, $0 \leq p \leq 1$.

$$p(k) = p(1 - p)^{k - 1}, k = 1, 2, \dots$$

$$\mu = \frac{1}{p}, \sigma^2 = \frac{1 - p}{p^2}$$

Poisson distribution

The number of events occurring in a fixed period of time t if these events occur with a known average rate κ and independently of the time since the last event is $\text{Po}(\lambda)$, $\lambda = t\kappa$.

$$p(k) = e^{-\lambda} \frac{\lambda^k}{k!}, k = 0, 1, 2, \dots$$

$$\mu = \lambda, \sigma^2 = \lambda$$

2.8.2 Continuous distributions

Uniform distribution

If the probability density function is constant between a and b and 0 elsewhere it is $\text{U}(a, b)$, $a < b$.

$$f(x) = \begin{cases} \frac{1}{b - a} & a < x < b \\ 0 & \text{otherwise} \end{cases}$$

$$\mu = \frac{a + b}{2}, \sigma^2 = \frac{(b - a)^2}{12}$$

Exponential distribution

The time between events in a Poisson process is $\text{Exp}(\lambda)$, $\lambda > 0$.

$$f(x) = \begin{cases} \lambda e^{-\lambda x} & x \geq 0 \\ 0 & x < 0 \end{cases}$$

$$\mu = \frac{1}{\lambda}, \sigma^2 = \frac{1}{\lambda^2}$$

Normal distribution

Most real random values with mean μ and variance σ^2 are well described by $\mathcal{N}(\mu, \sigma^2)$, $\sigma > 0$.

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x - \mu)^2}{2\sigma^2}}$$

If $X_1 \sim \mathcal{N}(\mu_1, \sigma_1^2)$ and $X_2 \sim \mathcal{N}(\mu_2, \sigma_2^2)$ then

$$aX_1 + bX_2 + c \sim \mathcal{N}(\mu_1 + \mu_2 + c, a^2\sigma_1^2 + b^2\sigma_2^2)$$

2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \dots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n \mathbf{p}^{(0)}$ is the probability distribution for X_n (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

π is a stationary distribution if $\pi = \pi \mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state i . π_j / π_i is the expected number of visits in state j between two visits in state i .

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, π_i is proportional to node i 's degree.

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k \rightarrow \infty} \mathbf{P}^k = \mathbf{1P}$.

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is j , is $a_{ij} = p_{ij} + \sum_{k \in \mathbf{G}} a_{ik} p_{kj}$. The expected time until absorption, when the initial state is i , is $t_i = 1 + \sum_{k \in \mathbf{G}} p_{ki} t_k$.

Data structures (3)

OrderStatisticTree.h

Description: A set (not multiset!) with support for finding the n'th For multiset, change less to less_equal, but lower_bound will become upper_bound and you can only erase iterator element, and finding the index of an element. To get a map, change null_type.

Time: $\mathcal{O}(\log N)$

819d08, 15 lines

```
using namespace __gnu_pbds;
```

```
template<class T>
using Tree = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;
```

```
void example() {
    Tree<int> t, t2; t.insert(8);
    auto it = t.insert(10).first;
    assert(it == t.lower_bound(9));
    assert(t.order_of_key(10) == 1);
    assert(t.order_of_key(11) == 2);
    assert(*t.find_by_order(0) == 8);
    t.join(t2); // assuming T < T2 or T > T2, merge t2 into t
}
```

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

140fd8, 6 lines

```
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
    const uint64_t C = (11)(4e18 * acos(0)) | 71;
    ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll, int, chash> h({}, {}, {}, {}, {
    1<<16});
```

Matrix.h

Description: Basic operations on square matrices.

Usage: Matrix<int, 3> A;
A.d = {{{{1, 2, 3}}, {{4, 5, 6}}, {{7, 8, 9}}}};
array<int, 3> vec = {1, 2, 3};
vec = (A*N) * vec;

6ab5db, 26 lines

```
template<class T, int N> struct Matrix {
    typedef Matrix M;
    array<array<T, N>, N> d{};
    M operator*(const M& m) const {
        M a;
        rep(i, 0, N) rep(j, 0, N)
            rep(k, 0, N) a.d[i][j] += d[i][k]*m.d[k][j];
        return a;
    }
    array<T, N> operator*(const array<T, N>& vec) const {
        array<T, N> ret{};
        rep(i, 0, N) rep(j, 0, N) ret[i] += d[i][j] * vec[j];
        return ret;
    }
};
```

```
    }
    M operator^(ll p) const {
        assert(p >= 0);
        M a, b(*this);
        rep(i, 0, N) a.d[i][i] = 1;
        while(p){
            if(p&1) a = a*b;
            b = b*b;
            p >>= 1;
        }
        return a;
    }
};
```

LineContainer.h
Description: Container where you can add lines of the form $kx+m$, and query maximum values at points x . Useful for dynamic programming (“convex hull trick”).
Time: $\mathcal{O}(\log N)$

Sec1c7, 30 lines

```
struct Line {
    mutable ll k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    // (for doubles, use inf = 1/.0, div(a, b) = a/b)
    static const ll inf = LLONG_MAX;
    ll div(ll a, ll b){ // floored division
        return a / b - ((a ^ b) < 0 && a % b); }
    bool isect(iterator x, iterator y){
        if(y == end()) return x->p = inf, 0;
        if(x->k == y->k) x->p = x->m > y->m ? inf : -inf;
        else x->p = div(y->m - x->m, x->k - y->k);
        return x->p >= y->p;
    }
    void add(ll k, ll m){
        auto z = insert({k, m, 0}), y = z++, x = y;
        while(isect(y, z)) z = erase(z);
        if(x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while((y = x) != begin() && (--x)->p >= y->p)
            isect(x, erase(y));
    }
    ll query(ll x){
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};
```

Lichao.h
Description: Li Chao tree on known x -coordinates; supports line/segment add.
Usage: `LiChao tr(xs); tr.add({m, b}); tr.add({m, b}, 1, r); tr.query(x);`
Time: $\mathcal{O}(\log N)$ per line/query, $\mathcal{O}(\log^2 N)$ per segment

23b102, 42 lines

```
struct Line {
    ll m=0, b=0;
    ll operator()(ll x) const { return m*x+b; }
};
struct LiChao {
    const ll INF = 4e18;
    vector<ll> x; vector<Line> st;
    bool mn;
    LiChao(vector<ll> x, bool mn=true) : x(x), mn(mn) {
        sort(all(this->x)); this->x.erase(unique(all(this->x)),
            this->x.end());
    }
```

```
        assert(sz(this->x));
        st.assign(4*sz(this->x), {0, mn?INF:-INF});
    }
    bool better(Line a, Line b, int p){ return mn ? a(x[p])<b(x[p]
        ) : a(x[p])>b(x[p]); }
    void add(Line f, int l, int r, int p){
        int m=(l+r)/2; bool a=better(f, st[p], l), b=better(f, st[p]
            l, m);
        if(b) swap(f, st[p]);
        if(r-l==1) return;
        if(a!=b) add(f, l, m, p*2); else add(f, m, r, p*2+1);
    }
    void add(Line f){ add(f, 0, sz(x), 1); }
    void add(Line f, int a, int b, int l, int r, int p){
        if(b<=l or r<=a) return;
        if(a<=l and r<=b) return add(f, l, r, p);
        int m=(l+r)/2;
        add(f, a, b, l, m, p*2); add(f, a, b, m, r, p*2+1);
    }
    void add(Line f, ll l, ll r){ // x in [l, r]
        int a=lower_bound(all(x), l)-x.begin(), b=upper_bound(all(x)
            , r)-x.begin();
        if(a<b) add(f, a, b, 0, sz(x), 1);
    }
    ll query(int p, int l, int r, int q){
        ll ans=st[p](x[q]);
        if(r-l==1) return ans;
        int m=(l+r)/2; ll z=q<m?query(p*2, l, m, q):query(p*2+1, m,
            r, q);
        return mn?min(ans, z):max(ans, z);
    }
    ll query(ll q){
        q=lower_bound(all(x), q)-x.begin(); assert(q<sz(x));
        return query(1, 0, sz(x), q);
    }
};
```

Treap.h
Description: A short self-balancing tree. It acts as a sequential container with log-time splits/joins, and is easy to augment with additional data.
Time: $\mathcal{O}(\log N)$

1754b4, 53 lines

```
struct Node {
    Node *l = 0, *r = 0;
    int val, y, c = 1;
    Node(int val) : val(val), y(rand()){ }
    void recalc();
};

int cnt(Node* n){ return n ? n->c : 0; }
void Node::recalc(){ c = cnt(l) + cnt(r) + 1; }

template<class F> void each(Node* n, F f){
    if(n){ each(n->l, f); f(n->val); each(n->r, f); }
}

pair<Node*, Node*> split(Node* n, int k){
    if(!n) return {};
    if(cnt(n->l) >= k){ // "n->val >= k" for lower_bound(k)
        auto [L, R] = split(n->l, k);
        n->l = R;
        n->recalc();
        return {L, n};
    }else{
        auto [L, R] = split(n->r, k - cnt(n->l) - 1); // and just "
            k"
        n->r = L;
        n->recalc();
        return {n, R};
    }
```

```
    }
}

Node* merge(Node* l, Node* r){
    if(!l) return r;
    if(!r) return l;
    if(l->y > r->y){
        l->r = merge(l->r, r);
        return l->recalc(), l;
    }else{
        r->l = merge(l, r->l);
        return r->recalc(), r;
    }
}

Node* ins(Node* t, Node* n, int pos){
    auto [l, r] = split(t, pos);
    return merge(merge(l, n), r);
}

// Example application: move the range [l, r] to index k
void move(Node*& t, int l, int r, int k){
    Node *a, *b, *c;
    tie(a, b) = split(t, l); tie(b, c) = split(b, r - l);
    if(k <= l) t = merge(ins(a, b, k), c);
    else t = merge(a, ins(c, b, k - r));
}
```

pre-seg.h
Description: Persistent segment tree: point assign and range sum on $[0, n]$.
Usage: `Seg st(n); int root2=st.modify(root1, p, x); st.query(root2, l, r);`
Time: $\mathcal{O}(\log n)$ per operation

b8993e, 27 lines

```
struct Seg {
    struct Node { int ch[2]{}; ll val=0; };
    vector<Node> t{};
    int n;
    Seg(int n, int q = 0) : n(n){ if(q) t.reserve(1+q*(__lg(n)+1)
        ); }
    int modify(int u, int l, int r, int p, ll x){
        int v = sz(t); t.pb(t[u]);
        if(r-l == 1) return t[v].val=x, v;
        int m=(l+r)/2, d=p>=m;
        if(d) t[v].ch[1]=modify(t[v].ch[1], m, r, p, x);
        else t[v].ch[0]=modify(t[v].ch[0], l, m, p, x);
        t[v].val=t[t[v].ch[0]].val+t[t[v].ch[1]].val;
        return v;
    }
    int modify(int root, int p, ll x){
        assert(0<=p and p<n); return modify(root, 0, n, p, x);
    }
    ll query(int u, int l, int r, int a, int b) const {
        if(!u or b<=l or r<=a) return 0;
        if(a<=l and r<=b) return t[u].val;
        int m=(l+r)/2;
        return query(t[u].ch[0], l, m, a, b)+query(t[u].ch[1], m, r
            , a, b);
    }
    ll query(int root, int l, int r) const {
        return query(root, 0, n, l, r);
    }
};
```

MoQueries.h
Description: Mo ordering for half-open intervals. add/del get (index, end).
Time: $\mathcal{O}(N\sqrt{Q})$ moves

6d40ed, 53 lines

```
template<class Add, class Del, class Calc>
```

```
auto mo(vector<pii> Q, Add add, Del del, Calc calc, int blk =350){
    using T=decltype(calc());
    int L=0, R=0; vi ord(sz(Q)); vector<T> ans(sz(Q));
    iota(all(ord), 0);
    sort(all(ord), [&](int i, int j){
        auto [a, b]=Q[i]; auto [c, d]=Q[j];
        return pii(a/blk, b^-(a/blk&1)) < pii(c/blk, d^-(c/blk&1));
    });
    for(int i:ord){
        auto [l, r]=Q[i];
        while(L>l) add(--L, 0);
        while(R<r) add(R++, 1);
        while(L<l) del(L++, 0);
        while(R>r) del(--R, 1);
        ans[i]=calc();
    }
    return ans;
}
```

```
template<class Add, class Del, class Calc>
auto moTree(vector<array<int, 2>> Q, vector<vi> &g, Add add, Del del,
            Calc calc, int root=0, int blk=350){
    using T=decltype(calc());
    int n=sz(g), ti=n, pos[2]={root, root};
    vi ord(sz(Q)), I(n*2), L(n), R(n), in(n), par(n), tmp(n);
    vector<T> ans(sz(Q)); add(root, 0), in[root]=1;
    auto dfs=[&](auto self, int u, int p, int d)->void {
        par[u]=p; L[u]=ti;
        if(d) I[u]=ti++;
        for(int v:g[u]) if(v!=p) self(self, v, u, !d);
        if(!d) I[u]=ti++;
        R[u]=ti;
    };
    dfs(dfs, root, -1, 0); iota(all(ord), 0);
    sort(all(ord), [&](int i, int j){
        auto a=Q[i], b=Q[j];
        return pii(I[a[0]]/blk, I[a[1]]^(I[a[0]]/blk&1)) <
            pii(I[b[0]]/blk, I[b[1]]^(I[b[0]]/blk&1));
    });
    for(int qi:ord) rep(e, 0, 2){
        int &a=pos[e], b=Q[qi][e], k=0;
        auto step=[&](int c){
            if(in[c]) del(a, e), in[a]=0; else add(c, e), in[c]=1;
            a=c;
        };
        while(not(L[b]<=L[a] and R[a]<=R[b])) tmp[k++]=b, b=par[b];
        while(a!=b) step(par[a]);
        while(k) step(tmp[--k]);
        if(e) ans[qi]=calc();
    }
    return ans;
}
```

Numerical (4)

4.1 Polynomials and recurrences

```
Polynomial.h
c9b7b0, 17 lines

struct Poly {
    vector<double> a;
    double operator()(double x) const {
        double val = 0;
        for(int i = sz(a); i--;) (val *= x) += a[i];
        return val;
    }
}
```

```
void diff(){
    rep(i, 1, sz(a)) a[i-1] = i*a[i];
    a.pop_back();
}

void divroot(double x0){
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
}
};
```

PolyRoots.h

Description: Finds the distinct real roots of a polynomial in [xmin, xmax]. Time: $\mathcal{O}(n^2 \log(1/\epsilon))$

```
"Polynomial.h"
7f59a3, 28 lines

vector<double> polyRoots(Poly p, double xmin, double xmax){
    while(sz(p.a) > 1 && abs(p.a.back()) < 1e-14) p.a.pop_back();
    if(sz(p.a) <= 1) return {};
    if(sz(p.a) == 2){
        double x = -p.a[0]/p.a[1];
        return xmin <= x && x <= xmax ? vector<double>{x} : vector<double>{};
    }
    Poly der = p; der.diff();
    auto at = polyRoots(der, xmin, xmax);
    at.insert(at.begin(), xmin); at.pb(xmax);
    vector<double> ret;
    auto add = [&](double x){
        if(abs(p(x)) < 1e-8 && (ret.empty() || abs(ret.back()-x) > 1e-7)) ret.pb(x);
    };
    rep(i, 0, sz(at)){
        add(at[i]);
        if(i+1 == sz(at)) break;
        double l = at[i], r = at[i+1], fl = p(l), fr = p(r);
        if((fl > 0) == (fr > 0) || abs(fl) < 1e-8 || abs(fr) < 1e-8) continue;
        rep(it, 0, 80){
            double m = (l+r)/2, fm = p(m);
            if((fl > 0) == (fm > 0)) l = m, fl = fm;
            else r = m;
        }
        add((l+r)/2);
    }
    return ret;
}
```

PolyInterpolate.h

Description: Given n points $(x[i], y[i])$, computes an $n-1$ -degree polynomial p that passes through them: $p(x) = a[0] * x^0 + \dots + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$. Time: $\mathcal{O}(n^2)$

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n){
    vd res(n), temp(n);
    rep(k, 0, n-1) rep(i, k+1, n)
        y[i] = (y[i] - y[k]) / (x[i] - x[k]);
    double last = 0; temp[0] = 1;
    rep(k, 0, n) rep(i, 0, n){
        res[i] += y[k] * temp[i];
        swap(last, temp[i]);
        temp[i] -= last * x[k];
    }
    return res;
}
```

BerlekampMassey.h

Description: Recovers any n -order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. Should work on any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$. Usage: berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2} Time: $\mathcal{O}(N^2)$

```
".../number-theory/ModPow.h"
96548b, 20 lines

vector<ll> berlekampMassey(vector<ll> s){
    int n = sz(s), L = 0, m = 0;
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    rep(i, 0, n){ ++m;
        ll d = s[i] % mod;
        rep(j, 1, L+1) d = (d + C[j] * s[i - j]) % mod;
        if(!d) continue;
        T = C; ll coef = d * modpow(b, mod-2) % mod;
        rep(j, m, n) C[j] = (C[j] - coef * B[j - m]) % mod;
        if(2 * L > i) continue;
        L = i + 1 - L; B = T; b = d; m = 0;
    }

    C.resize(L + 1); C.erase(C.begin());
    for(ll& x : C) x = (mod - x) % mod;
    return C;
}
```

LinearRecurrence.h

Description: Generates the k 'th term of an n -order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \dots n-1]$ and $tr[0 \dots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey. Usage: linearRec({0, 1}, {1, 1}, k) // k 'th Fibonacci number Time: $\mathcal{O}(n^2 \log k)$

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k){
    int n = sz(tr);

    auto combine = [&](Poly a, Poly b){
        Poly res(n * 2 + 1);
        rep(i, 0, n+1) rep(j, 0, n+1)
            res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
        for(int i = 2 * n; i > n; --i) rep(j, 0, n)
            res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    Poly pol(n + 1), e(pol);
    pol[0] = e[1] = 1;

    for(++k; k; k /= 2){
        if(k % 2) pol = combine(pol, e);
        e = combine(e, e);
    }

    ll res = 0;
    rep(i, 0, n) res = (res + pol[i + 1] * S[i]) % mod;
    return res;
}
```

Operation.h

Description: Formal power series modulo 998244353. Requires: NumberTheoreticTransform Time: $\mathcal{O}(N \log N)$ for Mul, $\mathcal{O}(N \log^2 N)$ for Newton operations

```
"NumberTheoreticTransform.h"
d8ac6c, 83 lines
```

```

struct FPS : vl {
    using vl::vl;
    FPS(const vl &a):vl(a){}
    int n() const { return sz(*this); }
    FPS cut(int m) const { FPS r=*this; r.resize(m); return r; }
    FPS& imul(ll x){
        x%=ntt_mod; if(x<0) x+=ntt_mod;
        for(ll &v:*this) v=v*x%ntt_mod;
        return *this;
    }
    FPS& iadd(const FPS &b, int s=1){
        if(n()<b.n()) resize(b.n());
        rep(i, 0, b.n()){
            (*this)[i]=((*this)[i]+s*b[i])%ntt_mod;
            if((*this)[i]<0) (*this)[i]+=ntt_mod;
        }
        return *this;
    }
    FPS Mul(const FPS &b) const { return FPS(conv(*this, b)); }
    FPS Inv(int m=-1) const {
        if(m<0) m=n();
        assert(n() and (*this)[0]);
        FPS r{nttPow((*this)[0], ntt_mod-2)};
        for(int k=2; k/2<m; k<=1){
            int z=min(k, m); FPS q=cut(z).Mul(r).cut(z).imul(-1);
            q[0]=(q[0]+2)%ntt_mod;
            r=r.Mul(q).cut(z);
        }
        return r.cut(m);
    }
    FPS Dx() const {
        if(n()<2) return {};
        FPS r(n()-1); rep(i, 1, n()) r[i-1]=(*this)[i]*i%ntt_mod;
        return r;
    }
    FPS Sx() const {
        static vl iv{0, 1}; int m=n()+1;
        while(sz(iv)<m){
            ll i=sz(iv); iv.pb(ntt_mod-ntt_mod/i*iv[ntt_mod%i]%
                ntt_mod);
        }
        FPS r(m); rep(i, 0, n()) r[i+1]=(*this)[i]*iv[i+1]%ntt_mod;
        return r;
    }
    FPS Ln(int m=-1) const {
        if(m<0) m=n();
        assert(n() and (*this)[0]==1);
        return Dx().Mul(Inv(m)).Sx().cut(m);
    }
    FPS Exp(int m=-1) const {
        if(m<0) m=n();
        assert(!n() or !(*this)[0]);
        FPS r{1};
        for(int k=2; k/2<m; k<=1){
            int z=min(k, m); FPS q=cut(z).iadd(r.Ln(z), -1);
            q[0]=(q[0]+1)%ntt_mod;
            r=r.Mul(q).cut(z);
        }
        return r.cut(m);
    }
    FPS Pow(ll k, int m=-1) const {
        if(m<0) m=n();
        if(!k){ FPS r(m); if(m) r[0]=1; return r; }
        int p=0; while(p<n() and !(*this)[p]) ++p;
        if(p==n() or (p and k>=(m+p-1)/p)) return FPS(m);
        int sh=(int)(p*k), z=m-sh; ll c=(*this)[p], ic=nttPow(c,
            ntt_mod-2);
        FPS f(begin()+p, end()); f=f.cut(z).imul(ic);
        f=f.Ln(z).imul(k%ntt_mod).Exp(z).imul(nttPow(c, k));

```

```

        f.insert(f.begin(), sh, 0); return f.cut(m);
    }
    pair<FPS, FPS> DivMod(FPS b) const {
        FPS a=*this;
        while(a.n()>1 and !a.back()) a.pop_back();
        while(b.n()>1 and !b.back()) b.pop_back();
        assert(b.n() and b.back());
        if(a.n()<b.n()) return {{0}, a};
        int m=a.n()-b.n()+1;
        FPS x=a, y=b; reverse(all(x)), reverse(all(y));
        FPS q=x.cut(m).Mul(y.cut(m).Inv(m)).cut(m); reverse(all(q))
            ;
        FPS r=a; r.iadd(b.Mul(q), -1); r.resize(max<int>(1, b.n()
            -1));
        return {q, r};
    }
};
using Poly_t=FPS;

```

4.2 Optimization

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is eps . Works equally well for maximization with a small change in the code. See Ternary-Search.h in the Various chapter for a discrete version.

Usage: double func(double x){ return 4+x+.3*x*x; }

double xmin = gss(-1000, 1000, func);

Time: $\mathcal{O}(\log((b-a)/\epsilon))$

31d45b, 14 lines

```

double gss(double a, double b, double (*f)(double)){
    double r = (sqrt(5)-1)/2, eps = 1e-7;
    double x1 = b - r*(b-a), x2 = a + r*(b-a);
    double f1 = f(x1), f2 = f(x2);
    while(b-a > eps)
        if(f1 < f2){ //change to > to find maximum
            b = x2; x2 = x1; f2 = f1;
            x1 = b - r*(b-a); f1 = f(x1);
        }else{
            a = x1; x1 = x2; f1 = f2;
            x2 = a + r*(b-a); f2 = f(x2);
        }
    return a;
}

```

HillClimbing.h

Description: Poor man's optimization for unimodal functions.

8eeef, 14 lines

typedef array<double, 2> P;

```

template<class F> pair<double, P> hillClimb(P start, F f){
    pair<double, P> cur(f(start), start);
    for(double jmp = 1e9; jmp > 1e-20; jmp /= 2){
        rep(j, 0, 100) rep(dx, -1, 2) rep(dy, -1, 2){
            P p = cur.second;
            p[0] += dx*jmp;
            p[1] += dy*jmp;
            cur = min(cur, make_pair(f(p), p));
        }
    }
    return cur;
}

```

Simplex.h

Description: Maximizes $c^T x$ subject to $Ax \leq b$ and $x \geq 0$.

Duality: min $b^T y$ subject to $A^T y \geq c$, $y \geq 0$; finite optima agree. Returns {value, x}; value is -inf if infeasible and inf if unbounded.

Time: $\mathcal{O}(NM * \#pivots)$, exponential in the worst case

1589b0, 65 lines

```

typedef double T;
typedef vector<T>vd;
typedef vector<vd>vvd;
const T eps = 1e-8, inf = 1 / .0;
#define MP make_pair
#define ltj(X) if(s == -1 or MP(X[j], N[j]) < MP(X[s], N[s])) s
    = j

struct LPSolver{
    int m, n;
    vi N, B;
    vvd D;
    LPSolver(const vvd &A, const vd &b, const vd &c):
        m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)){
        rep(i, 0, m) rep(j, 0, n) D[i][j] = A[i][j];
        rep(i, 0, m) B[i] = n + i, D[i][n] = -1, D[i][n+1] = b[i]
            ;
        rep(j, 0, n) N[j] = j, D[m][j] = -c[j];
        N[n] = -1;
        D[m+1][n] = 1;
    }
    void pivot(int r, int s){
        T *a = D[r].data(), inv = 1 / a[s];
        rep(i, 0, m+2) if(i != r and abs(D[i][s]) > eps){
            T *b = D[i].data(), inv2 = b[s] * inv;
            rep(j, 0, n+2) b[j] -= a[j] * inv2;
            b[s] = a[s] * inv2;
        }
        rep(j, 0, n+2) if(j != s) D[r][j] *= inv;
        rep(i, 0, m+2) if(i != r) D[i][s] *= -inv;
        D[r][s] = inv;
        swap(B[r], N[s]);
    }
    bool simplex(int phase){
        int x = m + phase - 1;
        for(;;){
            int s = -1;
            rep(j, 0, n+1) if(N[j] != -phase) ltj(D[x]);
            if(D[x][s] >= -eps) return 1;
            int r = -1;
            rep(i, 0, m){
                if(D[i][s] <= eps) continue;
                if(r == -1 or MP(D[i][n+1] / D[i][s], B[i]) <
                    MP(D[r][n+1] / D[r][s], B[r])) r = i;
            }
            if(r == -1) return 0;
            pivot(r, s);
        }
    }
    pair<T, vd> solve(){
        int r = 0;
        rep(i, 1, m) if(D[i][n+1] < D[r][n+1]) r = i;
        if(D[r][n+1] < -eps){
            pivot(r, n);
            if(!simplex(2) or D[m+1][n+1] < -eps) return {-inf, {
                }};
            rep(i, 0, m) if(B[i] == -1){
                int s = 0;
                rep(j, 1, n+1) ltj(D[i]);
                pivot(i, s);
            }
        }
        bool ok = simplex(1);
        vd x(n);
        rep(i, 0, m) if(B[i] < n) x[B[i]] = D[i][n+1];
        return {ok ? D[m][n+1] : inf, x};
    }
};

```

4.3 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}(N^3)$

```
double det(vector<vector<double>> a){
    int n = sz(a); double res = 1;
    rep(i, 0, n){
        int b = i;
        rep(j, i+1, n) if(fabs(a[j][i]) > fabs(a[b][i])) b = j;
        if(i != b) swap(a[i], a[b]), res *= -1;
        res *= a[i][i];
        if(res == 0) return 0;
        rep(j, i+1, n){
            double v = a[j][i] / a[i][i];
            if(v != 0) rep(k, i+1, n) a[j][k] -= v * a[i][k];
        }
    }
    return res;
}
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}(N^3)$

```
ll det(vector<vector<ll>> a, ll mod){
    int n = sz(a); ll ans = 1;
    for(auto &r : a) for(ll &x : r) x = (x%mod+mod)%mod;
    rep(i, 0, n){
        rep(j, i+1, n){
            while(a[j][i] != 0){ // gcd step
                ll t = a[i][i] / a[j][i];
                if(t){
                    rep(k, i, n) a[i][k] = (a[i][k]-a[j][k]*t)%mod;
                }
                swap(a[i], a[j]);
                ans *= -1;
            }
        }
        ans = ans * a[i][i] % mod;
        if(!ans) return 0;
    }
    return (ans + mod) % mod;
}
```

SolveLinear.h

Description: Solves $A*x=b$. Returns rank, x, or -1, if inconsistent.
Time: $\mathcal{O}(N^2M)$

```
typedef vector<double>vd;
const double eps = 1e-12;

pair<int, vd> solveLinear(vector<vd>A, vd b, int m = -1){
    int n = sz(A), rank = 0, br = 0, bc = 0;
    if(n) m = sz(A[0]);
    else if(m == -1) m = 0;
    vi col(m);
    iota(all(col), 0);
    rep(i, 0, n){
        double v, bv = 0;
        rep(r, i, n) rep(c, i, m) if((v = abs(A[r][c])) > bv)
            br = r, bc = c, bv = v;
        if(bv <= eps){
            rep(j, i, n) if(abs(b[j]) > eps) return {-1, {}};
            break;
        }
        swap(A[i], A[br]);
        swap(b[i], b[br]);
```

```
        swap(col[i], col[bc]);
        rep(j, 0, n) swap(A[j][i], A[j][bc]);
        bv = 1 / A[i][i];
        rep(j, i + 1, n){
            double z = A[j][i] * bv;
            b[j] -= z * b[i];
            rep(k, i + 1, m) A[j][k] -= z * A[i][k];
        }
        rank++;
    }
    vd x(m);
    for(int i = rank; i--){
        b[i] /= A[i][i];
        x[col[i]] = b[i];
        rep(j, 0, i) b[j] -= A[j][i] * b[i];
    }
    return {rank, x};
}
```

SolveLinear2.h

Description: Solves $A*x=b$. Undetermined $x[i]$ are NaN. Returns rank, x, or -1, if inconsistent.
Time: $\mathcal{O}(NMmin(N,M))$

```
"SolveLinear.h"
pair<int, vd> solveLinear2(vector<vd>A, vd b, int m = -1){
    int n = sz(A), r = 0;
    if(n) m = sz(A[0]);
    else if(m == -1) m = 0;
    vi col;
    rep(c, 0, m){
        if(r == n) break;
        int p = r;
        rep(i, r, n) if(abs(A[i][c]) > abs(A[p][c])) p = i;
        if(abs(A[p][c]) <= eps) continue;
        swap(A[p], A[r]);
        swap(b[p], b[r]);
        double z = A[r][c];
        rep(j, c, m) A[r][j] /= z;
        b[r] /= z;
        rep(i, 0, n) if(i != r){
            z = A[i][c];
            rep(j, c, m) A[i][j] -= z * A[r][j];
            b[i] -= z * b[r];
        }
        col.pb(c);
        r++;
    }
    rep(i, r, n) if(abs(b[i]) > eps) return {-1, {}};
    vd x(m, numeric_limits<double>::quiet_NaN());
    vector<bool>pivot(m);
    for(int c : col) pivot[c] = 1;
    rep(i, 0, r){
        bool unique = 1;
        rep(c, 0, m) if(!pivot[c] and abs(A[i][c]) > eps) unique = 0;
        if(unique) x[col[i]] = b[i];
    }
    return {r, x};
}
```

SolveLinearBinary.h

Description: Solves $A*x=b$ over F2. Returns rank, x, or -1, .
Time: $\mathcal{O}(N^2M)$

```
typedef bitset<1000>bs;

pair<int, bs> solveLinear(vector<bs>A, vi b, int m){
    int n = sz(A), rank = 0, br;
    assert(m <= sz(bs()));
```

```
    vi col(m);
    iota(all(col), 0);
    rep(i, 0, n){
        for(br = i; br < n; br++) if(A[br].any()) break;
        if(br == n){
            rep(j, i, n) if(b[j]) return {-1, {}};
            break;
        }
        int bc = A[br]._Find_next(i - 1);
        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        rep(j, 0, n) if(A[j][i] != A[j][bc]) A[j].flip(i), A[j].flip(bc);
        rep(j, i + 1, n) if(A[j][i]) b[j] ^= b[i], A[j] ^= A[i];
        rank++;
    }
    bs x;
    for(int i = rank; i--){
        if(!b[i]) continue;
        x[col[i]] = 1;
        rep(j, 0, i) b[j] ^= A[j][i];
    }
    return {rank, x};
}
```

MatrixInverse.h

Description: Invert matrix A . Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of $A \pmod p$, and k is doubled in each step.
Time: $\mathcal{O}(n^3)$

```
int matInv(vector<vector<double>>& A){
    int n = sz(A); vi col(n);
    vector<vector<double>> tmp(n, vector<double>(n));
    rep(i, 0, n) tmp[i][i] = 1, col[i] = i;

    rep(i, 0, n){
        int r = i, c = i;
        rep(j, i, n) rep(k, i, n)
            if(fabs(A[j][k]) > fabs(A[r][c]))
                r = j, c = k;
        if(fabs(A[r][c]) < 1e-12) return i;
        A[i].swap(A[r]); tmp[i].swap(tmp[r]);
        rep(j, 0, n)
            swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
        swap(col[i], col[c]);
        double v = A[i][i];
        rep(j, i+1, n){
            double f = A[j][i] / v;
            A[j][i] = 0;
            rep(k, i+1, n) A[j][k] -= f*A[i][k];
            rep(k, 0, n) tmp[j][k] -= f*tmp[i][k];
        }
        rep(j, i+1, n) A[i][j] /= v;
        rep(j, 0, n) tmp[i][j] /= v;
        A[i][i] = 1;
    }

    for(int i = n-1; i > 0; --i) rep(j, 0, i){
        double v = A[j][i];
        rep(k, 0, n) tmp[j][k] -= v*tmp[i][k];
    }

    rep(i, 0, n) rep(j, 0, n) A[col[i]][col[j]] = tmp[i][j];
    return n;
}
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i, \, 1 \leq i \leq n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, \dots, -1, 1\}, \{0, c_1, c_2, \dots, c_n\}, \{b_1, b_2, \dots, b_n, 0\}, \{a_0, d_1, d_2, \dots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.

If $|d_i| > |p_i| + |q_{i-1}|$ for all i , or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.

Time: $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b){
    int n = sz(b); vi tr(n);
    rep(i, 0, n-1){
        if(abs(diag[i]) < 1e-9 * abs(super[i])){ // diag[i] == 0
            b[i+1] -= b[i] * diag[i+1] / super[i];
            if(i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
            diag[i+1] = sub[i]; tr[++i] = 1;
        }else{
            diag[i+1] -= super[i]*sub[i]/diag[i];
            b[i+1] -= b[i]*sub[i]/diag[i];
        }
    }
    for(int i = n; i--;){
        if(tr[i]){
            swap(b[i], b[i-1]);
            diag[i-1] = diag[i];
            b[i] /= super[i-1];
        }else{
            b[i] /= diag[i];
            if(i) b[i-1] -= b[i]*super[i-1];
        }
    }
    return b;
}
```

4.4 Fourier transforms

FastFourierTransform.h

Description: `fft(a)` computes $\hat{f}(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k . N must be a power of 2. Useful for convolution: `conv(a, b) = c`, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by `n`, `reverse(start+1, end)`, FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16} ; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

00ced6, 35 lines

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a){
    int n = sz(a), L = 31 - __builtin_clz(n);
    static vector<complex<long double>> R(2, 1);
    static vector<C> rt(2, 1); // (^ 10% faster if double)
    for(static int k = 2; k < n; k *= 2){
        R.resize(n); rt.resize(n);
        auto x = polar(1.0L, acos(-1.0L) / k);
```

```
        rep(i, k, 2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
    }
    vi rev(n);
    rep(i, 0, n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
    rep(i, 0, n) if(i < rev[i]) swap(a[i], a[rev[i]]);
    for(int k = 1; k < n; k *= 2)
        for(int i = 0; i < n; i += 2 * k) rep(j, 0, k){
            C z = rt[j+k] * a[i+j+k]; // (25% faster if hand-rolled)
            a[i + j + k] = a[i + j] - z;
            a[i + j] += z;
        }
    }
    vd conv(const vd& a, const vd& b){
        if(a.empty() || b.empty()) return {};
        vd res(sz(a) + sz(b) - 1);
        int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
        vector<C> in(n), out(n);
        copy(all(a), begin(in));
        rep(i, 0, sz(b)) in[i].imag(b[i]);
        fft(in);
        for(C& x : in) x *= x;
        rep(i, 0, n) out[i] = in[-i & (n - 1)] - conj(in[i]);
        fft(out);
        rep(i, 0, sz(res)) res[i] = imag(out[i]) / (4 * n);
        return res;
    }
```

FastFourierTransformMod.h

Description: Exact convolution modulo M using three NTT primes and CRT. Supports transform lengths up to 2^{21} .

Time: $\mathcal{O}(N \log N)$

a6bf18, 62 lines

```
typedef vector<ll> vl;
namespace conv_mod {
template<int P> ll pw(ll a, ll n){
    ll r = 1;
    for(; n; n >>= 1, a = a*a%P) if(n&1) r = r*a%P;
    return r;
}
template<int P, int G> void ntt(vl& a, bool inv){
    int n = sz(a);
    for(int i = 1, j = 0; i < n; i++){
        int bit = n >> 1;
        for(; j&bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if(i < j) swap(a[i], a[j]);
    }
    for(int len = 2; len <= n; len <= 1){
        ll w1 = pw<P>(G, (P-1)/len);
        if(inv) w1 = pw<P>(w1, P-2);
        for(int i = 0; i < n; i += len){
            ll w = 1;
            rep(j, 0, len/2){
                ll u = a[i+j], v = a[i+j+len/2]*w%P;
                a[i+j] = (u+v)%P;
                a[i+j+len/2] = (u-v+P)%P;
                w = w*w1%P;
            }
        }
    }
    if(inv){
        ll ni = pw<P>(n, P-2);
        for(ll &x : a) x = x*ni%P;
    }
}
template<int P, int G> vl conv(const vl& a, const vl& b, int n,
    int s){
    vl x(n), y(n);
    rep(i, 0, sz(a)) x[i] = a[i]%P;
```

```
        rep(i, 0, sz(b)) y[i] = b[i]%P;
    ntt<P, G>(x, 0); ntt<P, G>(y, 0);
    rep(i, 0, n) x[i] = x[i]*y[i]%P;
    ntt<P, G>(x, 1); x.resize(s);
    return x;
}
}
```

```
template<int M> vl convMod(const vl& a, const vl& b){
    if(a.empty() || b.empty()) return {};
    const ll p0 = 998244353, p1 = 1004535809, p2 = 469762049;
    int s = sz(a)+sz(b)-1, n = 1;
    while(n < s) n <= 1;
    auto x = conv_mod::conv<p0, 3>(a, b, n, s);
    auto y = conv_mod::conv<p1, 3>(a, b, n, s);
    auto z = conv_mod::conv<p2, 3>(a, b, n, s);
    ll i01 = conv_mod::pw<p1>(p0, p1-2);
    ll p01 = p0*p1, i012 = conv_mod::pw<p2>(p01*p2, p2-2);
    vl ret(s);
    rep(i, 0, s){
        ll b1 = (y[i]-x[i]+p1)%p1*i01%p1;
        ll b2 = (z[i]-(x[i]+p0*b1)%p2+p2)%p2*i012%p2;
        ret[i] = (x[i]+(__int128)p0*b1+(__int128)p01*b2)%M;
    }
    return ret;
}
```

NumberTheoreticTransform.h

Description: NTT/convolution modulo 998244353, up to size 2^{23} .

Time: $\mathcal{O}(N \log N)$

25a2b8, 53 lines

```
const ll ntt_mod = 998244353, ntt_root = 62;
typedef vector<ll> vl;
ll nttPow(ll a, ll n){
    ll r = 1; for(; n; n >>= 1, a = a * a % ntt_mod) if(n & 1) r
        = r * a % ntt_mod;
    return r;
}

void nttRaw(vl &a, bool inv = false){
    int n = sz(a); static vl rt(2, 1), irt(2, 1);
    for(static int k = 2; k < n; k *= 2){
        rt.resize(2 * k); irt.resize(2 * k);
        ll z = nttPow(ntt_root, (ntt_mod - 1) / (2 * k)), iz =
            nttPow(z, ntt_mod - 2);
        rt[k] = irt[k] = 1;
        rep(i, 1, k) rt[k + i] = rt[k + i - 1] * z % ntt_mod,
            irt[k + i] = irt[k + i - 1] * iz % ntt_mod;
    }
    if(!inv) for(int k = n / 2; k; k /= 2)
        for(int i = 0; i < n; i += 2 * k) rep(j, 0, k){
            ll x = a[i + j], y = a[i + j + k];
            a[i + j] = x + y < ntt_mod ? x + y : x + y - ntt_mod;
            a[i + j + k] = (x - y + ntt_mod) * rt[k + j] % ntt_mod;
        }
    else{
        for(int k = 1; k < n; k *= 2)
            for(int i = 0; i < n; i += 2 * k) rep(j, 0, k){
                ll x = a[i + j], y = a[i + j + k] * irt[k + j] %
                    ntt_mod;
                a[i + j] = x + y < ntt_mod ? x + y : x + y - ntt_mod;
                a[i + j + k] = x - y < 0 ? x - y + ntt_mod : x - y;
            }
        ll z = nttPow(n, ntt_mod - 2); for(ll &x : a) x = x * z %
            ntt_mod;
    }
}
void ntt(vl &a){
```

```
int n = sz(a); assert(n and !(n & (n - 1)) and n <= (1 << 23));
);
nttRaw(a);
for(int i = 1, j = 0; i < n; i++){
    int b = n / 2; for(; j & b; b /= 2) j ^= b;
    j ^= b; if(i < j) swap(a[i], a[j]);
}
}

v1 conv(const v1 &a, const v1 &b){
    if(a.empty() or b.empty()) return {};
    if(min(a.size(), b.size()) <= 60){
        v1 c(sz(a) + sz(b) - 1);
        rep(i, 0, sz(a)) rep(j, 0, sz(b)) c[i + j] = (c[i + j] + a[i] * b[j]) % ntt_mod;
        return c;
    }
    int s = sz(a) + sz(b) - 1, n = 1; while(n < s) n *= 2;
    assert(n <= (1 << 23)); v1 x(a), y(b); x.resize(n); y.resize(n);
    nttRaw(x); nttRaw(y);
    rep(i, 0, n) x[i] = x[i] * y[i] % ntt_mod;
    nttRaw(x, true); x.resize(s); return x;
}
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x\oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two.

Time: $\mathcal{O}(N \log N)$

```
void FST(vi& a, bool inv, int mod = 0){
    for(int n = sz(a), step = 1; step < n; step *= 2){
        for(int i = 0; i < n; i += 2 * step) rep(j, i, i+step){
            int &u = a[j], &v = a[j + step], x = u, y = v;
            if(mod){
                if(inv){ u = y - x; if(u < 0) u += mod; v = x; }
                else { u = y; v = x + y; if(v >= mod) v -= mod; }
            }
            else tie(u, v) = inv ? pii(y - x, x) : pii(y, x + y); // AND
            inv ? pii(v, u - v) : pii(u + v, u); // OR
            pii(u + v, u - v); // XOR
        }
    }
    if(inv) for(int& x : a) x /= sz(a); // XOR only
}

vi conv(vi a, vi b, int mod = 0){
    FST(a, 0, mod); FST(b, 0, mod);
    rep(i, 0, sz(a)) a[i] = mod ? (int)((ll)a[i]*b[i] % mod) : a[i]*b[i];
    FST(a, 1, mod); return a;
}
```

Number theory (5)

5.1 Modular arithmetic

ModularArithmetic.h

Description: Operators for modular arithmetic. You need to set mod to some number first and then you can use the structure.

```
"euclid.h"
const ll mod = 17; // change to something else
struct Mod {
    ll x;
    Mod(ll y) : Mod(y % mod + mod, 0){}
    Mod(ll y, int) : x(y < mod ? y : y - mod){}
    Mod operator+(Mod b){ return {x + b.x, 0}; }
```

```
Mod operator-(Mod b){ return {x - b.x + mod, 0}; }
Mod operator*(Mod b){ return {x * b.x % mod, 0}; }
Mod operator/(Mod b){ return *this * invert(b); }
Mod invert(Mod a){
    ll x, y, g = euclid(a.x, mod, x, y);
    assert(g == 1); return x;
}
Mod operator^(ll e){
    if(!e) return 1;
    Mod r = *this ^ (e / 2); r = r * r;
    return e&1 ? *this * r : r;
}
```

ModInverse.h

Description: Inverses of $[1, n]$ modulo a prime, assuming $n \leq \text{mod}$.

Time: $\mathcal{O}(n)$

```
vector<ll> inverses(int n, ll mod){
    vector<ll> inv(n);
    if(n > 1) inv[1] = 1;
    rep(i, 2, n) inv[i] = mod - mod / i * inv[mod % i] % mod;
    return inv;
}
```

ModPow.h

```
const ll mod = 1000000007; // faster if const

ll modpow(ll b, ll e, ll m = mod){
    ll ans = 1;
    b %= m;
    for(; e; b = b * b % m, e /= 2)
        if(e & 1) ans = ans * b % m;
    return ans;
}
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod m$, or -1 if no such x exists. $\text{modLog}(a, 1, m)$ can be used to calculate the order of a .

Time: $\mathcal{O}(\sqrt{m})$

```
ll modLog(ll a, ll b, ll m){
    ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
    unordered_map<ll, ll> A;
    while(j <= n && (e = f = e * a % m) != b % m)
        A[e * b % m] = j++;
    if(e == b % m) return j;
    if(__gcd(m, e) == __gcd(m, b))
        rep(i, 2, n+2) if(A.count(e = e * f % m))
            return n * i - A[e];
    return -1;
}
```

ModSum.h

Description: Sums of mod'ed arithmetic progressions.

$\text{modsum}(\text{to}, c, k, m) = \sum_{i=0}^{\text{to}-1} (ki + c) \% m$. divsum is similar but for floored division.

Time: $\log(m)$, with a large constant.

```
typedef unsigned long long ull;
ull sumsq(ull to){ return to / 2 * ((to-1) | 1); }
```

```
ull divsum(ull to, ull c, ull k, ull m){
    ull res = k / m * sumsq(to) + c / m * to;
    k %= m; c %= m;
    if(!k) return res;
    ull to2 = (to * k + c) / m;
    return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}
```

```
}

ll modsum(ull to, ll c, ll k, ll m){
    c = ((c % m) + m) % m;
    k = ((k % m) + m) % m;
    return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

ModMulLL.h

Description: Portable $a*b \pmod M$ for $0 \leq a, b < M < 2^{64}$.

Time: $\mathcal{O}(1)$ for modmul , $\mathcal{O}(\log e)$ for modpow

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M){
    return (ull)((unsigned __int128)a * b % M);
}
ull modpow(ull a, ull e, ull M){
    ull r = 1;
    for(; e; e >= 1, a = modmul(a, a, M))
        if(e & 1) r = modmul(r, a, M);
    return r;
}
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod p$ ($-x$ gives the other solution).

Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p

```
"ModPow.h"
ll sqrt(ll a, ll p){
    a %= p; if(a < 0) a += p;
    if(a == 0) return 0;
    assert(modpow(a, (p-1)/2, p) == 1); // else no solution
    if(p % 4 == 3) return modpow(a, (p+1)/4, p);
    // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
    ll s = p - 1, n = 2;
    int r = 0, m;
    while(s % 2 == 0)
        ++r, s /= 2;
    while(modpow(n, (p - 1) / 2, p) != p - 1) ++n;
    ll x = modpow(a, (s + 1) / 2, p);
    ll b = modpow(a, s, p), g = modpow(n, s, p);
    for(;; r = m){
        ll t = b;
        for(m = 0; m < r && t != 1; ++m)
            t = t * t % p;
        if(m == 0) return x;
        ll gs = modpow(g, 1LL << (r - m - 1), p);
        g = gs * gs % p;
        x = x * gs % p;
        b = b * g % p;
    }
}
```

5.2 Primality

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.

Time: $\text{LIM} = 1e9 \approx 1.5s$

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes(){
    const int S = (int)round(sqrt(LIM)), R = LIM / 2;
    vi pr = {2}, sieve(S+1); pr.reserve((size_t)(LIM/log(LIM) * 1.1));
    vector<pii> cp;
    for(int i = 3; i <= S; i += 2) if(!sieve[i]){
        cp.push_back({i, i * i / 2});
        for(int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
    }
```

```
}
for(int L = 1; L <= R; L += S){
    array<bool, S> block{};
    for(auto &[p, idx] : cp)
        for(int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i, 0, min(S, R - L))
        if(!block[i]) pr.push_back((L + i) * 2 + 1);
}
for(int i : pr) isPrime[i] = 1;
return pr;
}
```

MillerRabin.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.
Time: 7 times the complexity of $a^b \bmod c$.

"ModMuLL.h"	60dcd1, 12 lines
-------------	------------------

```
bool isPrime(ull n){
    if(n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
    ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
        s = __builtin_ctzll(n-1), d = n >> s;
    for(ull a : A){ // ^ count trailing zeroes
        ull p = modpow(a%n, d, n), i = s;
        while(p != 1 && p != n - 1 && a % n && i--){
            p = modmul(p, p, n);
            if(p != n-1 && i != s) return 0;
        }
        return 1;
    }
}
```

Factor.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).
Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMuLL.h", "MillerRabin.h"	d8d98d, 18 lines
------------------------------	------------------

```
ull pollard(ull n){
    ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
    auto f = [&](ull x){ return modmul(x, x, n) + i; };
    while(t++ % 40 || __gcd(prd, n) == 1){
        if(x == y) x = ++i, y = f(x);
        if((q = modmul(prd, max(x, y) - min(x, y), n))) prd = q;
        x = f(x), y = f(f(y));
    }
    return __gcd(prd, n);
}

vector<ull> factor(ull n){
    if(n == 1) return {};
    if(isPrime(n)) return {n};
    ull x = pollard(n);
    auto l = factor(x), r = factor(n / x);
    l.insert(l.end(), all(r));
    return l;
}
```

5.3 Divisibility

euclid.h

Description: Finds two integers x and y , such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in `__gcd` instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

	33ba8f, 5 lines
--	-----------------

```
ll euclid(ll a, ll b, ll &x, ll &y){
    if(!b) return x = 1, y = 0, a;
    ll d = euclid(b, a % b, y, x);
    return y -= a/b * x, d;
}
```

CRT.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m, x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \leq x < \operatorname{lcm}(m, n)$. Assumes $mn < 2^{62}$.

"euclid.h"	04d93a, 7 lines
------------	-----------------

```
ll crt(ll a, ll m, ll b, ll n){
    if(n > m) swap(a, b), swap(m, n);
    ll x, y, g = euclid(m, n, x, y);
    assert((a - b) % g == 0); // else no solution
    x = (b - a) % n * x % n / g * m + a;
    return x < 0 ? x + m*n/g : x;
}
```

5.3.1 Bézout’s identity

For $a \neq, b \neq 0$, then $d = \gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If (x, y) is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

Description: Euler’s ϕ function is defined as $\phi(n) := \#$ of positive integers $\leq n$ that are coprime with n . $\phi(1) = 1, p \text{ prime} \Rightarrow \phi(p^k) = (p - 1)p^{k-1}, m, n \text{ coprime} \Rightarrow \phi(mn) = \phi(m)\phi(n)$. If $n = p_1^{k_1}p_2^{k_2} \dots p_r^{k_r}$ then $\phi(n) = (p_1 - 1)p_1^{k_1-1} \dots (p_r - 1)p_r^{k_r-1}$. $\phi(n) = n \cdot \prod_{p|n} (1 - 1/p)$.
 $\sum_{d|n} \phi(d) = n, \sum_{1 \leq k \leq n, \gcd(k, n)=1} k = n\phi(n)/2, n > 1$
Euler’s thm: $a, n \text{ coprime} \Rightarrow a^{\phi(n)} \equiv 1 \pmod n$.
Fermat’s little thm: $p \text{ prime} \Rightarrow a^{p-1} \equiv 1 \pmod p \ \forall a$.

	cf7d6d, 8 lines
--	-----------------

```
const int LIM = 5000000;
int phi[LIM];

void calculatePhi(){
    rep(i, 0, LIM) phi[i] = i&1 ? i : i/2;
    for(int i = 3; i < LIM; i += 2) if(phi[i] == i)
        for(int j = i; j < LIM; j += i) phi[j] -= phi[j] / i;
}
```

5.4 Fractions

ContinuedFractions.h

Description: Given N and a real number $x \geq 0$, finds the closest rational approximation p/q with $p, q \leq N$. It will obey $|p/q - x| \leq 1/qN$.
For consecutive convergents, $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$. (p_k/q_k alternates between $> x$ and $< x$.) If x is rational, y eventually becomes ∞ ; if x is the root of a degree 2 polynomial the a ’s eventually become cyclic.
Time: $\mathcal{O}(\log N)$

	dd6c5e, 21 lines
--	------------------

```
typedef double d; // for N ~ 1e7; long double for N ~ 1e9
pair<ll, ll> approximate(d x, ll N){
    ll LP = 0, LQ = 1, P = 1, Q = 0, inf = LLONG_MAX; d y = x;
    for(;;){
        ll lim = min(P ? (N-LP) / P : inf, Q ? (N-LQ) / Q : inf),
            a = (ll)floor(y), b = min(a, lim),
            NP = b*P + LP, NQ = b*Q + LQ;
        if(a > b){
            // If b > a/2, we have a semi-convergent that gives us a
            // better approximation; if b = a/2, we *may* have one.
            // Return {P, Q} here for a more canonical approximation.
            return (abs(x - (d)NP / (d)NQ) < abs(x - (d)P / (d)Q)) ?
```

```
        make_pair(NP, NQ) : make_pair(P, Q);
    }
    if(abs(y = 1/(y - (d)a)) > 3*N){
        return {NP, NQ};
    }
    LP = P; P = NP;
    LQ = Q; Q = NQ;
}
}
```

FracBinarySearch.h

Description: Given f and N , finds the smallest fraction $p/q \in [0, 1]$ such that $f(p/q)$ is true, and $p, q \leq N$. You may want to throw an exception from f if it finds an exact solution, in which case N can be removed.
Usage: fracBS({}(Frac f){ return f.p>=3*f.q; }, 10); // {1, 3}
Time: $\mathcal{O}(\log(N))$

	27ab3e, 25 lines
--	------------------

```
struct Frac { ll p, q; };

template<class F>
Frac fracBS(F f, ll N){
    bool dir = 1, A = 1, B = 1;
    Frac lo{0, 1}, hi{1, 1}; // Set hi to 1/0 to search (0, N]
    if(f(lo)) return lo;
    assert(f(hi));
    while(A || B){
        ll adv = 0, step = 1; // move hi if dir, else lo
        for(int si = 0; step; (step *= 2) >= si){
            adv += step;
            Frac mid{lo.p * adv + hi.p, lo.q * adv + hi.q};
            if(abs(mid.p) > N || mid.q > N || dir == !f(mid)){
                adv -= step; si = 2;
            }
        }
        hi.p += lo.p * adv;
        hi.q += lo.q * adv;
        dir = !dir;
        swap(lo, hi);
        A = B; B = !!adv;
    }
    return dir ? hi : lo;
}
```

5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2), \quad b = k \cdot (2mn), \quad c = k \cdot (m^2 + n^2),$$

with $m > n > 0, k > 0, m \perp n$, and either m or n even.

5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power p^a , except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^\times$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

5.7 Estimates

$$\sum_{d|n} d = \mathcal{O}(n \log \log n).$$

The number of divisors of n is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d)g(n/d)$$

Other useful formulas/forms:

$$\sum_{d|n} \mu(d) = [n = 1] \text{ (very useful)}$$

$$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n)g(d)$$

$$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m)g(\lfloor \frac{n}{m} \rfloor)$$

Combinatorial (6)

6.1 Permutations

6.1.1 Factorial

<i>n</i>	1	2	3	4	5	6	7	8	9	10
<i>n</i> !	1	2	6	24	120	720	5040	40320	362880	3628800
<i>n</i>	11	12	13	14	15	16	17			
<i>n</i> !	4.0e7	4.8e8	6.2e9	8.7e10	1.3e12	2.1e13	3.6e14			
<i>n</i>	20	25	30	40	50	100	150	171		
<i>n</i> !	2e18	2e25	3e32	8e47	3e64	9e157	6e262	>DBL_MAX		

IntPerm.h
Description: Permutation -> integer conversion. (Not order preserving.)
Integer -> permutation can use a lookup table.
Time: $\mathcal{O}(n)$

```
int permToInt(vi& v){
    int use = 0, i = 0, r = 0;
    for(int x:v) r = r * ++i + __builtin_popcount(use & -(1<<x)),
        use |= 1 << x;
    return r;
}
```

6.1.2 Cycles

Let $g_S(n)$ be the number of n -permutations whose cycle lengths all belong to the set S . Then

$$\sum_{n=0}^\infty g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1) + D(n-2)) = nD(n-1) + (-1)^n = \left\lfloor \frac{n!}{e} \right\rfloor$$

6.1.4 Burnside’s lemma

Given a group G of symmetries and a set X , the number of elements of X up to symmetry equals

$$\frac{1}{|G|} \sum_{g \in G} |X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts “configurations” (of some sort) of length n , we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n} \sum_{k=0}^{n-1} f(\gcd(n, k)) = \frac{1}{n} \sum_{k|n} f(k) \phi(n/k).$$

6.2 Partitions and subsets

6.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1, \quad p(n) = \sum_{k \in \mathbb{Z} \setminus \{0\}} (-1)^{k+1} p(n - k(3k - 1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

<i>n</i>	0	1	2	3	4	5	6	7	8	9	20	50	100
<i>p</i> (<i>n</i>)	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

6.2.2 Lucas’ Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \dots + n_1 p + n_0$ and $m = m_k p^k + \dots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^k \binom{n_i}{m_i} \pmod p$.

6.2.3 Binomials

multinomial.h

Description: Computes $\binom{k_1 + \dots + k_n}{k_1, k_2, \dots, k_n} = \frac{(\sum k_i)!}{k_1! k_2! \dots k_n!}$.

```
ll multinomial(vi& v){
    ll c = 1, m = v.empty() ? 1 : v[0];
    rep(i, 1, sz(v)) rep(j, 0, v[i]) c = c * ++m / (j+1);
    return c;
}
```

6.3 General purpose numbers

6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).

$$B[0, \dots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \dots]$$

Sums of powers:

$$\sum_{i=1}^n i^m = \frac{1}{m+1} \sum_{k=0}^m \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\begin{aligned} \sum_{i=m}^\infty f(i) &= \int_m^\infty f(x) dx - \sum_{k=1}^\infty \frac{B_k}{k!} f^{(k-1)}(m) \\ &\approx \int_m^\infty f(x) dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m)) \end{aligned}$$

6.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n, k) = c(n-1, k-1) + (n-1)c(n-1, k), \quad c(0, 0) = 1$$
$$\sum_{k=0}^n c(n, k) x^k = x(x+1) \dots (x+n-1)$$

$$c(8, k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$$
$$c(n, 2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$$

6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j :s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j :s s.t. $\pi(j) \geq j$, k j :s s.t. $\pi(j) > j$.

$$E(n, k) = (n-k)E(n-1, k-1) + (k+1)E(n-1, k)$$

$$E(n, 0) = E(n, n-1) = 1$$

$$E(n, k) = \sum_{j=0}^k (-1)^j \binom{n+1}{j} (k+1-j)^n$$

6.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n, k) = S(n-1, k-1) + kS(n-1, k)$$

$$S(n, 1) = S(n, n) = 1$$

$$S(n, k) = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$$

6.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

6.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i : $n_1 n_2 \dots n_k n^{k-2}$
with degrees d_i : $(n-2)! / ((d_1-1)! \dots (d_n-1)!)$

6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$
$$C_0 = 1, \; C_{n+1} = \frac{2(2n+1)}{n+2} C_n, \; C_{n+1} = \sum C_i C_{n-i}$$
$$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \dots$$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n + 1$ leaves (0 or 2 children).
- ordered trees with $n + 1$ vertices.
- ways a convex polygon with $n + 2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

Graph (7)

7.1 Fundamentals

BellmanFord.h
Description: Single-source shortest paths with negative cycles. Unreachable vertices have INF; vertices after a negative cycle have -INF.
Time: $\mathcal{O}(VE)$

```
struct BellmanFord{
    struct Edge{ int u, v; ll w; };
    const ll INF = 1LL << 62;
    int n;
    vector<Edge>ed;
    BellmanFord(int N) : n(N){}
    void addEdge(int u, int v, ll w){
        ed.pb({u, v, w});
    }
    pair<vector<ll>, vi> calc(int s){
        vector<ll>d(n, INF);
        vi par(n, -1);
        d[s] = 0;
        rep(i, 0, n){
            for(auto [u, v, w] : ed) if(d[u] != INF){
                ll nd = max(-INF, d[u] + w);
                if(nd < d[v]) d[v] = i == n - 1 ? -INF : nd, par[v] = u;
            }
        }
        rep(i, 0, n) for(auto [u, v, w] : ed)
            if(d[u] == -INF) d[v] = -INF;
        return {d, par};
    }
};
```

FloydWarshall.h
Description: Calculates all-pairs shortest path in a directed graph that might have negative edge weights. Input is an distance matrix m , where $m[i][j] = \text{inf}$ if i and j are not adjacent. As output, $m[i][j]$ is set to the shortest distance between i and j , inf if no path, or -inf if the path goes through a negative-weight cycle.
Time: $\mathcal{O}(N^3)$

```
const ll inf = 1LL << 62;
void floydWarshall(vector<vector<ll>>& m){
    int n = sz(m);
    rep(i, 0, n) m[i][i] = min(m[i][i], 0LL);
```

```
    rep(k, 0, n) rep(i, 0, n) rep(j, 0, n)
        if(m[i][k] != inf && m[k][j] != inf){
            auto newDist = max(m[i][k] + m[k][j], -inf);
            m[i][j] = min(m[i][j], newDist);
        }
    rep(k, 0, n) if(m[k][k] < 0) rep(i, 0, n) rep(j, 0, n)
        if(m[i][k] != inf && m[k][j] != inf) m[i][j] = -inf;
}
```

7.2 Network flow

PushRelabel.h
Description: Push-relabel using the highest label selection rule and the gap heuristic. Quite fast in practice. To obtain the actual flow, look at positive values only.
Time: $\mathcal{O}(V^2\sqrt{E})$

```
struct PushRelabel {
    struct Edge {
        int dest, back;
        ll f, c;
    };
    vector<vector<Edge>> g;
    vector<ll> ec;
    vector<Edge*> cur;
    vector<vi> hs; vi H;
    PushRelabel(int n) : g(n), ec(n), cur(n), hs(2*n), H(n){

        void addEdge(int s, int t, ll cap, ll rcap=0){
            if(s == t) return;
            g[s].push_back({t, sz(g[t]), 0, cap});
            g[t].push_back({s, sz(g[s])-1, 0, rcap});
        }

        void addFlow(Edge& e, ll f){
            Edge &back = g[e.dest][e.back];
            if(!ec[e.dest] && f) hs[H[e.dest]].push_back(e.dest);
            e.f += f; e.c -= f; ec[e.dest] += f;
            back.f -= f; back.c += f; ec[back.dest] -= f;
        }

        ll calc(int s, int t){
            int v = sz(g); H[s] = v; ec[t] = 1;
            vi co(2*v); co[0] = v-1;
            rep(i, 0, v) cur[i] = g[i].data();
            for(Edge& e : g[s]) addFlow(e, e.c);

            for(int hi = 0;;){
                while(hs[hi].empty()) if(!hi--) return -ec[s];
                int u = hs[hi].back(); hs[hi].pop_back();
                while(ec[u] > 0) // discharge u
                    if(cur[u] == g[u].data() + sz(g[u])){
                        H[u] = 1e9;
                        for(Edge& e : g[u]) if(e.c && H[u] > H[e.dest]+1)
                            H[u] = H[e.dest]+1, cur[u] = &e;
                        if(++co[H[u]],!--co[hi] && hi < v)
                            rep(i, 0, v) if(hi < H[i] && H[i] < v)
                                --co[H[i]], H[i] = v + 1;
                        hi = H[u];
                    } else if(cur[u]->c && H[u] == H[cur[u]->dest]+1)
                        addFlow(*cur[u], min(ec[u], cur[u]->c));
                    else ++cur[u];
            }
        }

        bool leftOfMinCut(int a){ return H[a] >= sz(g); }
};
```

MinCostMaxFlowCostScaling.h

Description: Min-cost max-flow using Dinic followed by epsilon cost-scaling push-relabel. Supports negative costs and negative-cost cycles. Costs are multiplied by N internally; make sure this and the final answer fit in ll.
Usage: Prefer for large total flow, many unit-capacity edges, integer costs, negative edges/cycles, or min-cost circulation. SSP is usually faster for only a few augmentations. Not suitable for floating-point costs. For pure circulation, start with any feasible flow and run only the cost-scaling phase; do not invent s, t or run maxFlow.
Time: $\mathcal{O}(V^2E + VE\log(VC))$, where C is the maximum absolute cost

```
const ll CS_INF = numeric_limits<ll>::max() / 4;
struct CostScalingMCMF{
    struct edge{
        int from, to, rev;
        ll cap, cost, flow;
    };
    int N;
    vector<vector<edge>>ed;
    vector<ll>excess, pi;
    vi ptr, inq;
    CostScalingMCMF(int N) : N(N), ed(N), excess(N), pi(N), ptr(N), inq(N){}
    // Supports negative edges/cycles with finite capacities.
    // No negative-cost self-loops or unbounded instances.
    void addEdge(int from, int to, ll cap, ll cost){
        if(from == to){
            assert(cost >= 0);
            return;
        }
        assert(-CS_INF / N <= cost && cost <= CS_INF / N);
        cost *= N;
        ed[from].push_back(edge{from, to, sz(ed[to]), cap, cost, 0})
        ed[to].push_back(edge{to, from, sz(ed[from]) - 1, 0, -cost, 0});
    }
    ll residual(edge &e){
        return e.cap - e.flow;
    }
    void push(edge &e, ll fl){
        e.flow += fl;
        ed[e.to][e.rev].flow -= fl;
        excess[e.from] -= fl;
        excess[e.to] += fl;
    }
    bool admissible(edge &e){
        return residual(e) > 0 && pi[e.from] + e.cost - pi[e.to] < 0;
    }
    void relabel(int v, ll eps){
        ll best = -CS_INF;
        ptr[v] = -1;
        rep(i, 0, sz(ed[v])){
            edge &e = ed[v][i];
            if(residual(e) > 0 && best < pi[e.to] - e.cost){
                best = pi[e.to] - e.cost;
                ptr[v] = i;
            }
        }
        assert(ptr[v] != -1);
        pi[v] = best - eps;
    }
    void refine(ll eps){
        fill(all(excess), 0);
        fill(all(ptr), 0);
        fill(all(inq), 0);
```

```
rep(i, 0, N){
    for(edge &e : ed[i]){
        if(admissible(e)) push(e, residual(e));
    }
}
queue<int> q;
rep(i, 0, N){
    if(excess[i] > 0){
        q.push(i);
        inq[i] = 1;
    }
}
while(!q.empty()){
    int v = q.front();
    q.pop();
    inq[v] = 0;
    while(excess[v] > 0){
        if(ptr[v] == sz(ed[v])) relabel(v, eps);
        edge &e = ed[v][ptr[v]];
        if(!admissible(e)){
            ++ptr[v];
            continue;
        }
        ll fl = min(excess[v], residual(e));
        int to = e.to;
        push(e, fl);
        if(excess[to] > 0 && !inq[to]){
            q.push(to);
            inq[to] = 1;
        }
    }
}
}
}
ll maxFlow(int s, int t){
    vi level(N), work(N);
    auto bfs = [&]() {
        fill(all(level), -1);
        queue<int> q;
        level[s] = 0;
        q.push(s);
        while(!q.empty()){
            int v = q.front();
            q.pop();
            for(edge &e : ed[v]){
                if(residual(e) > 0 && level[e.to] == -1){
                    level[e.to] = level[v] + 1;
                    q.push(e.to);
                }
            }
        }
        return level[t] != -1;
    };
    auto dfs = [&](auto &&self, int v, ll pushed) -> ll {
        if(v == t or pushed == 0) return pushed;
        for(int &i = work[v]; i < sz(ed[v]); ++i){
            edge &e = ed[v][i];
            if(level[e.to] != level[v] + 1) continue;
            ll fl = self(self, e.to, min(pushed, residual(e)));
            if(fl == 0) continue;
            e.flow += fl;
            ed[e.to][e.rev].flow -= fl;
            return fl;
        }
        return 0;
    };
    ll flow = 0;
    while(bfs()){
        fill(all(work), 0);
        while(ll fl = dfs(dfs, s, CS_INF)) flow += fl;
```

```
    }
    return flow;
}
pair<ll, ll> maxflow(int s, int t){
    // Find any max flow, then reduce cost by circulation.
    ll totflow = maxFlow(s, t);
    ll eps = 0;
    rep(i, 0, N){
        for(edge &e : ed[i]){
            eps = max(eps, abs(e.cost));
        }
    }
    while(eps > 0){
        refine(eps);
        if(eps == 1) break;
        eps = max(1LL, eps / 4);
    }
    __int128 totcost = 0;
    rep(i, 0, N){
        for(edge &e : ed[i]){
            totcost += (__int128)e.cost * e.flow;
        }
    }
    return {totflow, (ll)(totcost / 2 / N)};
};
```

Dinic.h
Description: Flow algorithm with complexity $O(VE \log U)$ where $U = \max |cap|$. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{VE})$ for bipartite matching.

d79769, 42 lines

```
struct Dinic{
    struct Edge{
        int to, rev;
        ll c, oc;
        ll flow(){ return max(oc - c, 0LL); } // if you need flows
    };
    vi lvl, ptr, q;
    vector<vector<Edge>>adj;
    Dinic(int n) : lvl(n), ptr(n), q(n), adj(n){}
    void addEdge(int a, int b, ll c, ll rcap = 0){
        adj[a].push_back({b, sz(adj[b]), c, c});
        adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
    }
    ll dfs(int v, int t, ll f){
        if(v == t || !f) return f;
        for(int &i = ptr[v]; i < sz(adj[v]); i++){
            Edge &e = adj[v][i];
            if(lvl[e.to] == lvl[v] + 1)
                if(ll p = dfs(e.to, t, min(f, e.c))){
                    e.c -= p, adj[e.to][e.rev].c += p;
                    return p;
                }
        }
        return 0;
    }
    ll calc(int s, int t){
        ll flow = 0; q[0] = s;
        rep(L, 0, 31) do { // 'int L=30' maybe faster for random
            data
            lvl = ptr = vi(sz(q));
            int qi = 0, qe = lvl[s] = 1;
            while(qi < qe and !lvl[t]){
                int v = q[qi++];
                for(Edge e : adj[v])
                    if(!lvl[e.to] and e.c >> (30 - L))
                        q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
            }
        }
```

```
        while(ll p = dfs(s, t, LLONG_MAX)) flow += p;
    }while(lvl[t]);
    return flow;
}
bool leftOfMinCut(int a){ return lvl[a] != 0; }
};
```

MinCut.h
Description: After running max-flow, the left side of a min-cut from s to t is given by all vertices reachable from s , only traversing edges with positive residual capacity.

GlobalMinCut.h
Description: Find a global minimum cut in an undirected graph, as represented by an adjacency matrix.
Time: $O(V^3)$

8b0e19, 21 lines

```
pair<int, vi> globalMinCut(vector<vi> mat){
    pair<int, vi> best = {INT_MAX, {}};
    int n = sz(mat);
    vector<vi> co(n);
    rep(i, 0, n) co[i] = {i};
    rep(ph, 1, n){
        vi w = mat[0];
        size_t s = 0, t = 0;
        rep(it, 0, n-ph){ //  $O(V^2) \rightarrow O(E \log V)$  with prio. queue
            w[t] = INT_MIN;
            s = t, t = max_element(all(w)) - w.begin();
            rep(i, 0, n) w[i] += mat[t][i];
        }
        best = min(best, {w[t] - mat[t][t], co[t]});
        co[s].insert(co[s].end(), all(co[t]));
        rep(i, 0, n) mat[s][i] += mat[t][i];
        rep(i, 0, n) mat[i][s] = mat[s][i];
        mat[0][t] = INT_MIN;
    }
    return best;
};
```

GomoryHu.h
Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.
Time: $O(V)$ Flow Computations

0418b3, 13 lines

```
"PushRelabel.h"
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed){
    vector<Edge> tree;
    vi par(N);
    rep(i, 1, N){
        PushRelabel D(N); // Dinic also works
        for(Edge t : ed) D.addEdge(t[0], t[1], t[2], t[2]);
        tree.push_back({i, par[i], D.calc(i, par[i])});
        rep(j, i+1, N)
            if(par[j] == par[i] && D.leftOfMinCut(j)) par[j] = i;
    }
    return tree;
};
```

7.3 Matching
hopcroftKarp.h
Description: Maximum bipartite matching. $r[v]$ is the left match of right v .
Time: $O(E \sqrt{V})$

f50330, 40 lines

```
struct HopcroftKarp{
    int n, m;
```

```
vector<vi>g;
vi l, r, d, q;
HopcroftKarp(int N, int M): n(N), m(M), g(N), q(N){
void addEdge(int u, int v){
    g[u].pb(v);
}
bool dfs(int u){
    int z = exchange(d[u], 0) + 1;
    for(int v : g[u]){
        if(r[v] == -1 or (d[r[v]] == z and dfs(r[v]))){
            l[u] = v;
            r[v] = u;
            return 1;
        }
    }
    return 0;
}
pair<int, vi> calc(){
    int ans = 0;
    l.assign(n, -1);
    r.assign(m, -1);
    for(;;){
        int s = 0, t = 0;
        bool found = 0;
        d.assign(n, 0);
        rep(i, 0, n) if(l[i] == -1) q[t++] = i, d[i] = 1;
        while(s < t){
            int u = q[s++];
            for(int v : g[u]){
                if(r[v] == -1) found = 1;
                else if(!d[r[v]]) d[r[v]] = d[u] + 1, q[t++] = r[v];
            }
        }
        if(!found) return {ans, r};
        rep(i, 0, n) if(l[i] == -1) ans += dfs(i);
    }
}
};
```

DFSMatching.h

Description: Simple bipartite matching. r[v] is the left match of right v.

Time: $\mathcal{O}(VE)$

2847a4, 29 lines

```
struct DFSMatching{
    int n, m;
    vector<vi>g;
    vi r, vis;
    DFSMatching(int N, int M): n(N), m(M), g(N){
    void addEdge(int u, int v){
        g[u].pb(v);
    }
    bool dfs(int u){
        if(vis[u]) return 0;
        vis[u] = 1;
        for(int v : g[u]){
            if(r[v] == -1 or dfs(r[v])){
                r[v] = u;
                return 1;
            }
        }
        return 0;
    }
}
pair<int, vi> calc(){
    int ans = 0;
    r.assign(m, -1);
    rep(i, 0, n){
        vis.assign(n, 0);
        ans += dfs(i);
    }
}
```

```
        return {ans, r};
    }
};

MinimumVertexCover.h
Description: Minimum vertex cover in a bipartite graph. Returns left vertices as u and right vertices as n+v.
Time:  $\mathcal{O}(Esqrt(V))$ 
"hopcroftKarp.h"
99da56, 23 lines
struct MinimumVertexCover : HopcroftKarp{
    using HopcroftKarp::HopcroftKarp;
    vi cover(){
        auto [cnt, mt] = calc();
        vector<bool>left(n, 1), seen(m);
        for(int u : mt) if(u != -1) left[u] = 0;
        vi q, ans;
        rep(i, 0, n) if(left[i]) q.pb(i);
        while(sz(q)){
            int u = q.back();
            q.pop_back();
            left[u] = 1;
            for(int v : g[u]) if(!seen[v] and mt[v] != -1){
                seen[v] = 1;
                q.pb(mt[v]);
            }
        }
        rep(i, 0, n) if(!left[i]) ans.pb(i);
        rep(i, 0, m) if(seen[i]) ans.pb(n + i);
        assert(sz(ans) == cnt);
        return ans;
    }
};
```

WeightedMatching.h

Description: Minimum-cost matching of every row to a distinct column. N<=M.

Time: $\mathcal{O}(N^2M)$

feca40, 21 lines

```
template<class T> pair<T, vi> hungarian(const vector<vector<T>>
    &C){
    int i = sz(C), m = i ? sz(C[0]) : 0, c = 0, s, r;
    vector<T> dist(m), pot(m); vi match(i), rev(m, -1), cols(m),
        pre(m);
    T d = 0, nd, cost = 0;
    while(i--){
        rep(j, 0, m) dist[j] = C[i][j], cols[j] = j, pre[j] = i;
        for(s = 0;;){
            rep(j, s, m){
                c = cols[j], nd = dist[c] - pot[c];
                if(j == s or d > nd) d = nd, swap(cols[s], cols[j]);
            }
            if((r = rev[c = cols[s++]]) == -1) break;
            rep(j, 0, m) if(dist[j] > (nd = C[r][j] - C[r][c] + dist[c]))
                dist[j] = nd, pre[j] = r;
        }
        cost += dist[c];
        while(s--) pot[cols[s]] = dist[cols[s]] - d;
        for(; r != i; swap(c, match[r])) r = rev[c] = pre[c];
    }
    return {cost, match};
}
```

GeneralMatching.h

Description: Maximum matching in a general undirected graph.

Time: $\mathcal{O}(N^3)$

ef29b7, 72 lines

```
struct GeneralMatching {
    int n, st;
```

```
vector<vi> g;
vi mt, even, groupBuf;
vector<pii> nx;
GeneralMatching(int n) : n(n), g(n), mt(n), even(n), groupBuf(n), nx(n){}
void addEdge(int a, int b){
    g[a].pb(b); g[b].pb(a);
}
int group(int x){
    if(groupBuf[x] == -1 || even[groupBuf[x]] != st) return groupBuf[x];
    return groupBuf[x] = group(groupBuf[x]);
}
void rematch(int p, int b){
    int d = mt[p]; mt[p] = b;
    if(d == -1 || mt[d] != p) return;
    if(nx[p].S == -1){
        mt[d] = nx[p].F;
        rematch(nx[p].F, d);
    }else{
        rematch(nx[p].F, nx[p].S);
        rematch(nx[p].S, nx[p].F);
    }
}
bool augment(){
    even[st] = st; groupBuf[st] = -1; nx[st] = {-1, -1};
    queue<int> q; q.push(st);
    while(!q.empty()){
        int a = q.front(); q.pop();
        for(int b : g[a]){
            if(b == st) continue;
            if(mt[b] == -1){
                mt[b] = a; rematch(a, b);
                return 1;
            }
        }
        if(even[b] == st){
            int x = group(a), y = group(b), z = -1;
            if(x == y) continue;
            while(x != -1 || y != -1){
                if(y != -1) swap(x, y);
                if(nx[x] == pii(a, b)){
                    z = x;
                    break;
                }
                nx[x] = {a, b};
                x = group(nx[mt[x]].F);
            }
            for(int v : {group(a), group(b)}){
                while(v != z){
                    q.push(v); even[v] = st; groupBuf[v] = z;
                    v = group(nx[mt[v]].F);
                }
            }
        }else if(even[mt[b]] != st){
            even[mt[b]] = st; nx[b] = {-1, -1};
            nx[mt[b]] = {a, -1}; groupBuf[mt[b]] = b;
            q.push(mt[b]);
        }
    }
}
return 0;
}
pair<int, vi> calc(){
    fill(all(mt), -1); fill(all(even), -1); fill(all(groupBuf), 0);
    int ans = 0;
    rep(i, 0, n){
        st = i;
        if(mt[i] == -1) ans += augment();
    }
}
```

```
    }
    return {ans, mt};
}
};
```

7.4 Matroids

7.5 DFS algorithms

SCC.h
Description: Iterative Kosaraju SCC. sccid is in reverse topological order.
Time: $\mathcal{O}(V + E)$

```
struct SCC{
    int n, cnt = 0;
    vector<vi>g, rg;
    vi sccid;
    SCC(int n): n(n), g(n), rg(n), sccid(n, -1){}
    void add_edge(int u, int v){
        g[u].pb(v);
        rg[v].pb(u);
    }
    int build(){
        vi it(n), order, st;
        vector<bool>vis(n);
        rep(s, 0, n) if(!vis[s]){
            st.pb(s);
            vis[s] = 1;
            while(sz(st)){
                int u = st.back();
                if(it[u] == sz(g[u])){
                    order.pb(u);
                    st.pop_back();
                }else{
                    int v = g[u][it[u]++];
                    if(!vis[v]) vis[v] = 1, st.pb(v);
                }
            }
        }
        cnt = 0;
        sccid.assign(n, -1);
        reverse(all(order));
        for(int s : order) if(sccid[s] == -1){
            st.clear();
            st.pb(s);
            sccid[s] = cnt;
            while(sz(st)){
                int u = st.back();
                st.pop_back();
                for(int v : rg[u]) if(sccid[v] == -1) sccid[v] = cnt, st.pb(v);
            }
            cnt++;
        }
        for(int &x : sccid) x = cnt - 1 - x;
        return cnt;
    }
};
```

EBCC.h
Description: Bridges and edge-biconnected components of an undirected graph.
Time: $\mathcal{O}(V + E)$

```
struct EBCC{
    int n, m = 0, nbcc = 0, ti = 0;
    vector<vector<pii>>g;
    vi in, low, id, is_bridge;
    EBCC(int n): n(n), g(n){}
    void add_edge(int u, int v){
```

```
        g[u].pb({v, m});
        g[v].pb({u, m++});
    }
    int build(){
        nbcc = ti = 0; in.assign(n, -1); low.resize(n); id.assign(n, -1);
        is_bridge.assign(m, 0);
        vi it(n), par(n, -1), pe(n, -1), stk;
        rep(s, 0, n) if(in[s] == -1){
            in[s] = low[s] = ti++; stk.pb(s);
            while(sz(stk)){
                int u = stk.back();
                if(it[u] < sz(g[u])){
                    auto [v, e] = g[u][it[u]++];
                    if(e == pe[u]) continue;
                    if(in[v] == -1){
                        par[v] = u; pe[v] = e; in[v] = low[v] = ti++; stk.pb(v);
                    }else low[u] = min(low[u], in[v]);
                }else{
                    stk.pop_back();
                    if(pe[u] != -1){
                        int p = par[u];
                        low[p] = min(low[p], low[u]);
                        if(low[u] > in[p]) is_bridge[pe[u]] = 1;
                    }
                }
            }
        }
        rep(s, 0, n) if(id[s] == -1){
            id[s] = nbcc++; stk.pb(s);
            while(sz(stk)){
                int u = stk.back(); stk.pop_back();
                for(auto [v, e] : g[u]) if(!is_bridge[e] and id[v] == -1){
                    id[v] = id[s]; stk.pb(v);
                }
            }
        }
        return nbcc;
    }
};
```

Dominatortree.h
Description: Immediate dominators from source s. Unreachable nodes get -1.
Time: $\mathcal{O}((V + E)\alpha(V))$

```
struct DominatorTree {
    int n, id;
    vector <vector <int>> g, rg, bucket;
    vector <int> sdом, dom, vis, rev, pa, rt, mn, res;
    // dom[s] = s, dom[v] = -1 if s -> v not exists
    int query(int v, int x){
        if(rt[v] == v) return x ? -1 : v;
        int p = query(rt[v], 1);
        if(p == -1) return x ? rt[v] : mn[v];
        if(sdom[mn[v]] > sdom[mn[rt[v]]])
            mn[v] = mn[rt[v]];
        rt[v] = p;
        return x ? p : mn[v];
    }
    void dfs(int v){
        vis[v] = id, rev[id] = v;
        rt[id] = mn[id] = sdom[id] = id, id++;
        for(int u : g[v]){
            if(vis[u] == -1) dfs(u), pa[vis[u]] = vis[v];
            rg[vis[u]].pb(vis[v]);
        }
    }
```

```
    }
    void build(int s){
        id = 0;
        for(auto &v : rg) v.clear();
        for(auto &v : bucket) v.clear();
        fill(all(vis), -1); fill(all(dom), -1);
        dfs(s);
        for(int i = id - 1; ~i; --i){
            for(int u : rg[i])
                sdom[i] = min(sdom[i], sdom[query(u, 0)]);
            if(i) bucket[sdom[i]].pb(i);
            for(int u : bucket[i]){
                int p = query(u, 0);
                dom[u] = sdom[p] == i ? i : p;
            }
            if(i) rt[i] = pa[i];
        }
        fill(all(res), -1);
        for(int i = 1; i < id; ++i){
            if(dom[i] != sdom[i]) dom[i] = dom[dom[i]];
        }
        for(int i = 1; i < id; ++i)
            res[rev[i]] = rev[dom[i]];
        res[s] = s;
        for(int i = 0; i < n; ++i) dom[i] = res[i];
    }
    void add_edge(int u, int v){ g[u].pb(v); }
    DominatorTree (int _n) : n(_n), id(0), g(n), rg(n),
        bucket(n), sdom(n), dom(n, -1), vis(n, -1),
        rev(n), pa(n), rt(n), mn(n), res(n){}
};
```

BiconnectedComponents.h
Description: Vertex-biconnected components and the block-cut forest.
Original vertices are [0, n), component nodes are [n, n+nbcc).
Time: $\mathcal{O}(V + E)$

```
struct BCC{
    int n, m = 0, nbcc = 0, ti = 0;
    vector<vector<pii>>g;
    vector<vi>tree, comp;
    vi in, low;
    BCC(int n): n(n), g(n){}
    void add_edge(int u, int v){
        g[u].pb({v, m});
        g[v].pb({u, m++});
    }
    void add_comp(int u, int v, vi &stk){
        int id = n + nbcc++;
        tree.pb({}); comp.pb({});
        while(1){
            int x = stk.back(); stk.pop_back();
            tree[id].pb(x), tree[x].pb(id), comp.back().pb(x);
            if(x == v) break;
        }
        tree[id].pb(u), tree[u].pb(id), comp.back().pb(u);
    }
    void build(){
        nbcc = ti = 0; in.assign(n, -1); low.resize(n);
        tree.assign(n, {}); comp.clear(); tree.reserve(n + m);
        vi it(n), par(n, -1), pe(n, -1), stk, dfs;
        rep(s, 0, n) if(in[s] == -1){
            if(g[s].empty()){
                int id = n + nbcc++;
                tree.pb({s}); tree[s].pb(id); comp.pb({s});
                in[s] = low[s] = ti++;
                continue;
            }
            in[s] = low[s] = ti++; stk.pb(s); dfs.pb(s);
```

```
while(sz(dfs)){
    int u = dfs.back();
    if(it[u] < sz(g[u])){
        auto [v, e] = g[u][it[u]++];
        if(e == pe[u]) continue;
        if(in[v] == -1){
            par[v] = u; pe[v] = e; in[v] = low[v] = ti++;
            stk.pb(v); dfs.pb(v);
        }else low[u] = min(low[u], in[v]);
    }else{
        dfs.pop_back();
        if(pe[u] != -1){
            int p = par[u];
            low[p] = min(low[p], low[u]);
            if(low[u] >= in[p]) add_comp(p, u, stk);
        }
    }
    stk.clear();
}
}
bool is_cut(int u) const { return sz(tree[u]) > 1; }
const vi &bcc(int id) const { return comp[id]; }
};
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c, ... to a 2-SAT problem, so that an expression of the type $(a||b)&&(!a||c)&&(d||!b)&&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).

Usage: TwoSat ts(number of boolean variables);
ts.either(0, ~3); // Var 0 is true or var 3 is false
ts.setValue(2); // Var 2 is true
ts.atMostOne({0, ~1, 2}); // <= 1 of vars 0, ~1 and 2 are true
ts.solve(); // Returns true iff it is solvable
ts.values[0..N-1] holds the assigned values to the vars

Time: $\mathcal{O}(N + E)$, where N is the number of boolean variables, and E is the number of clauses.

b33304, 65 lines

```
struct TwoSat {
    int N;
    vector<vi> gr;
    vi values; // 0 = false, 1 = true

    TwoSat(int n = 0) : N(n), gr(2*n){

        int addVar(){ // (optional)
            gr.emplace_back();
            gr.emplace_back();
            return N++;
        }

        void either(int f, int j){
            f = max(2*f, -1-2*f);
            j = max(2*j, -1-2*j);
            gr[f].push_back(j^1);
            gr[j].push_back(f^1);
        }

        void setValue(int x){ either(x, x); }

        void atMostOne(const vi& li){ // (optional)
            if(sz(li) <= 1) return;
            int cur = ~li[0];
            rep(i, 2, sz(li)){
                int next = addVar();
                either(cur, ~li[i]);
                either(cur, next);
                either(~li[i], next);
                cur = ~next;
            }
        }
    };
};
```

```

    }
    either(cur, ~li[1]);
}

bool solve(){
    vector<vi> rg(2*N); vi vis(2*N), ord, comp(2*N, -1);
    rep(i, 0, 2*N) for(int j : gr[i]) rg[j].pb(i);
    rep(s, 0, 2*N) if(!vis[s]){
        vector<pii> st = {{s, 0}}; vis[s] = 1;
        while(!st.empty()){
            int u = st.back().F;
            if(st.back().S < sz(gr[u])){
                int v = gr[u][st.back().S++];
                if(!vis[v]) vis[v] = 1, st.pb({v, 0});
            }else ord.pb(u), st.pop_back();
        }
    }
    int c = 0;
    for(int oi = sz(ord); oi--){
        int s = ord[oi]; if(comp[s] != -1) continue;
        vi st = {s}; comp[s] = c;
        while(!st.empty()){
            int u = st.back(); st.pop_back();
            for(int v : rg[u]) if(comp[v] == -1) comp[v] = c, st.pb(v);
        }
        ++c;
    }
    values.resize(N);
    rep(i, 0, N){
        if(comp[2*i] == comp[2*i+1]) return 0;
        values[i] = comp[2*i] < comp[2*i+1];
    }
    return 1;
}
};
```

7.6 Coloring

EdgeColoring.h

Description: Given a simple, undirected graph with max degree D , computes a $(D + 1)$ -coloring of the edges such that no neighboring edges share a color. (D -coloring is NP-hard, but can be done for bipartite graphs by repeated matchings of max-degree nodes.)

Time: $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N, vector<pii> eds){
    vi cc(N + 1), ret(sz(eds)), fan(N), free(N), loc;
    for(pii e : eds) ++cc[e.first], ++cc[e.second];
    int u, v, ncols = *max_element(all(cc)) + 1;
    vector<vi> adj(N, vi(ncols, -1));
    for(pii e : eds){
        tie(u, v) = e;
        fan[0] = v;
        loc.assign(ncols, 0);
        int at = u, end = u, d, c = free[u], ind = 0, i = 0;
        while(d = free[v], !loc[d] && (v = adj[u][d]) != -1)
            loc[d] = ++ind, cc[ind] = d, fan[ind] = v;
        cc[loc[d]] = c;
        for(int cd = d; at != -1; cd ^= c ^ d, at = adj[at][cd])
            swap(adj[at][cd], adj[end = at][cd ^ c ^ d]);
        while(adj[fan[i]][d] != -1){
            int left = fan[i], right = fan[++i], e = cc[i];
            adj[u][e] = left;
            adj[left][e] = u;
            adj[right][e] = -1;
            free[right] = e;
        }
        adj[u][d] = fan[i];
        adj[fan[i]][d] = u;
    }
};
```

```
for(int y : {fan[0], u, end})
    for(int& z = free[y] = 0; adj[y][z] != -1; z++);
}
rep(i, 0, sz(eds))
    for(tie(u, v) = eds[i]; adj[u][ret[i]] != v;) ++ret[i];
return ret;
}
```

7.7 Heuristics

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.

Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs

b0d5b1, 12 lines

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}){
    if(!P.any()){ if(!X.any()) f(R); return; }
    auto q = (P | X)._Find_first();
    auto cands = P & ~eds[q];
    rep(i, 0, sz(eds)) if(cands[i]){
        R[i] = 1;
        cliques(eds, f, P & eds[i], X & eds[i], R);
        R[i] = P[i] = 0; X[i] = 1;
    }
}
```

MaximumClique.h

Description: Fast maximum clique for a symmetric bitset graph, $n \leq 200$.

b52fes, 43 lines

```
typedef vector<bitset<200>> vb;
struct MaxClique {
    double limit=.025, pk=0;
    struct Vertex { int i, d=0; };
    using vv=vector<Vertex>;
    vb e; vv V; vector<vi> C; vi qmax, q, steps, old;
    void init(vv &r){
        if(r.empty()) return;
        for(auto &v:r) v.d=0;
        for(auto &v:r) for(auto u:r) v.d+=e[v.i][u.i];
        sort(all(r), [](auto a, auto b){return a.d>b.d;});
        int mx=r[0].d;
        rep(i, 0, sz(r)) r[i].d=min(i, mx)+1;
    }
    void expand(vv &R, int lev=1){
        steps[lev] += steps[lev-1] - old[lev]; old[lev] = steps[lev-1];
        while(sz(R)){
            if(sz(q)+R.back().d<=sz(qmax)) return;
            q.pb(R.back().i); vv T;
            for(auto v:R) if(e[R.back().i][v.i]) T.pb({v.i});
            if(sz(T)){
                if((double)steps[lev] / (++pk<limit) init(T);
                int j=0, mx=1, mn=max<int>(sz(qmax)-sz(q)+1, 1);
                C[1].clear(), C[2].clear();
                for(auto v:T){
                    int k=1;
                    while(any_of(all(C[k]), [&](int i){return e[v.i][i];}
                        )) ++k;
                    if(k>mx) mx=k, C[mx+1].clear();
                    if(k<mn) T[j++].i=v.i;
                    C[k].pb(v.i);
                }
                if(j) T[j-1].d=0;
                rep(k, mn, mx+1) for(int i:C[k]) T[j].i=i, T[j++].d=k;
                expand(T, lev+1);
            } else if(sz(q)>sz(qmax)) qmax=q;
            q.pop_back(), R.pop_back();
        }
    }
};
```

```

    if (cur.rev) set_rev(c);
  }
  cur.rev = false;
}

void up(int u){
  cur.sum = o[lc].sum + cur.v + o[rc].sum;
  cur.rsum = o[rc].rsum + cur.v + o[lc].rsum;
  cur.sub = cur.vir + o[lc].sub + o[rc].sub + cur.sv;
  cur.size = o[lc].size + o[rc].size + 1;
}

void set_rev(int u){
  swap(lc, rc), swap(cur.sum, cur.rsum);
  cur.rev ^= 1;
}

// == PART HASH ==
void rotate(int u){
  int f = cur.pa, g = o[f].pa, l = is_rch(u);
  if (cur.ch[l ^ 1]) o[cur.ch[l ^ 1]].pa = f;
  if (not is_root(f)) o[g].ch[is_rch(f)] = u;
  o[f].ch[l] = cur.ch[l ^ 1], cur.ch[l ^ 1] = f;
  cur.pa = g, o[f].pa = u; up(f);
}

vector<int> stk;
void splay(int u){
  stk.clear(); stk.pb(u);
  while (not is_root(stk.back()))
    stk.push_back(o[stk.back()][l].pa);
  while (not stk.empty())
    down(stk.back()), stk.pop_back();
  for (int f = cur.pa; not is_root(u); f = cur.pa){
    if (!is_root(f))
      rotate(is_rch(u) == is_rch(f) ? f : u);
    rotate(u);
  }
  up(u);
}

void access(int x){
  for (int u = x, last = 0; u; u = cur.pa){
    splay(u);
    cur.vir = cur.vir + o[rc].sub - o[last].sub;
    rc = last; up(last = u);
  }
  splay(x);
}

int find_root(int u){
  int la = 0;
  for (access(u); u; u = lc) down(la = u);
  splay(la);
  return la;
}

void split(int x, int y){ chroot(x); access(y); }
void chroot(int u){ access(u); set_rev(u); }

// == PART HASH ==
LCT(int n = 0) : o(n + 1){ o[0].size = 0; }
void set_val(int u, const Val &v){
  access(u); cur.v = v; up(u); }
void set_sval(int u, const SVal &v){
  access(u); cur.sv = v; up(u); }
Val query(int x, int y){
  split(x, y); return o[y].sum; }
SVal subtree(int p, int u){
  chroot(p); access(u); return cur.vir + cur.sv; }
bool connected(int u, int v){
  return find_root(u) == find_root(v); }
void link(int x, int y){
  chroot(x); assert(find_root(y) != x); access(y);
  o[y].vir = o[y].vir + o[x].sub;
  up(o[x].pa = y);
}

```

```
void cut(int x, int y){
    split(x, y); assert(o[y].ch[0] == x && !o[x].ch[1]);
    o[y].ch[0] = o[x].pa = 0; up(y); }
#undef cur
#undef lc
#undef rc
};
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h"

39e620, 60 lines

```
struct Edge { int a, b; ll w; };
struct Node {
    Edge key;
    Node *l, *r;
    ll delta;
    void prop(){
        key.w += delta;
        if(l) l->delta += delta;
        if(r) r->delta += delta;
        delta = 0;
    }
    Edge top(){ prop(); return key; }
};
Node *merge(Node *a, Node *b){
    if(!a || !b) return a ?: b;
    a->prop(), b->prop();
    if(a->key.w > b->key.w) swap(a, b);
    swap(a->l, (a->r = merge(b, a->r)));
    return a;
}
void pop(Node*& a){ a->prop(); a = merge(a->l, a->r); }
```

```
pair<ll, vi> dmst(int n, int r, vector<Edge>& g){
    RollbackUF uf(n);
    vector<Node*> heap(n);
    for(Edge e : g) heap[e.b] = merge(heap[e.b], new Node(e));
    ll res = 0;
    vi seen(n, -1), path(n), par(n);
    seen[r] = r;
    vector<Edge> Q(n), in(n, {-1, -1}), comp;
    deque<tuple<int, int, vector<Edge>>> cyscs;
    rep(s, 0, n){
        int u = s, qi = 0, w;
        while(seen[u] < 0){
            if(!heap[u]) return {-1, {}};
            Edge e = heap[u]->top();
            heap[u]->delta -= e.w, pop(heap[u]);
            Q[qi] = e, path[qi++] = u, seen[u] = s;
            res += e.w, u = uf.find(e.a);
            if(seen[u] == s){
                Node* cyc = 0;
                int end = qi, time = uf.time();
                do cyc = merge(cyc, heap[w = path[--qi]]);
                while(uf.join(u, w));
                u = uf.find(u), heap[u] = cyc, seen[u] = -1;
                cyscs.push_front({u, time, {&Q[qi], &Q[end]}});
            }
        }
        rep(i, 0, qi) in[uf.find(Q[i].b)] = Q[i];
    }
}
```

```
for(auto& [u, t, comp] : cyscs){ // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for(auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
```

```
    }
    rep(i, 0, n) par[i] = in[i].a;
    return {res, par};
}
```

7.9 Math

7.9.1 Number of Spanning Trees

Create an $N \times N$ matrix mat , and for each edge $a \rightarrow b \in G$, do $\text{mat}[a][b]--$, $\text{mat}[b][b]++$ (and $\text{mat}[b][a]--$, $\text{mat}[a][a]++$ if G is undirected). Remove the i th row and column and take the determinant; this yields the number of directed spanning trees rooted at i (if G is undirected, remove any row/column).

7.9.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \dots \geq d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^k d_i \leq k(k-1) + \sum_{i=k+1}^n \min(d_i, k).$$

Geometry (8)

8.1 Geometric primitives

Point.h

Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

47ec0a, 28 lines

```
template <class T> int sgn(T x){ return (x > 0) - (x < 0); }
template<class T>
struct Point {
    typedef Point P;
    T x, y;
    explicit Point(T x=0, T y=0) : x(x), y(y){}
    bool operator<(P p) const { return tie(x, y) < tie(p.x, p.y); }
    bool operator==(P p) const { return tie(x, y)==tie(p.x, p.y); }
    P operator+(P p) const { return P(x+p.x, y+p.y); }
    P operator-(P p) const { return P(x-p.x, y-p.y); }
    P operator*(T d) const { return P(x*d, y*d); }
    P operator/(T d) const { return P(x/d, y/d); }
    T dot(P p) const { return x*p.x + y*p.y; }
    T cross(P p) const { return x*p.y - y*p.x; }
    T cross(P a, P b) const { return (a-*this).cross(b-*this); }
    T dist2() const { return x*x + y*y; }
    double dist() const { return sqrt((double)dist2()); }
    // angle to x-axis in interval [-pi, pi]
    double angle() const { return atan2(y, x); }
    P unit() const { return *this/dist(); } // makes dist()==1
    P perp() const { return P(-y, x); } // rotates +90 degrees
    P normal() const { return perp().unit(); }
    // returns point rotated 'a' radians ccw around the origin
    P rotate(double a) const {
        return P(x*cos(a)-y*sin(a), x*sin(a)+y*cos(a)); }
    friend ostream& operator<<(ostream& os, P p){
        return os << "(" << p.x << ", " << p.y << ")"; }
};
```

lineDistance.h

Description:

Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. $a==b$ gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

"Point.h"

53630a, 4 lines

```
template<class P>
double lineDist(P a, P b, P p){
    return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

SegmentDistance.h

Description:

Returns the shortest distance between point p and the line segment from point s to e.

Usage: Point<double> a, b(2, 2), p(1, 1);
bool onSegment = segDist(a, b, p) < 1e-10;

"Point.h"

d5f39f, 6 lines

```
typedef Point<double> P;
double segDist(P s, P e, P p){
    if(s==e) return (p-s).dist();
    auto d = (e-s).dist2(), t = min(d, max(.0, (p-s).dot(e-s)));
    return ((p-s)*d-(e-s)*t).dist()/d;
}
```

SegmentIntersection.h

Description:

If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.

Usage: vector<P> inter = segInter(s1, e1, s2, e2);
if(sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;

"Point.h", "OnSegment.h"

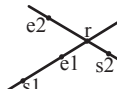
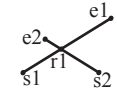
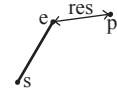
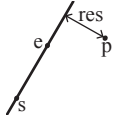
9d5f2f, 13 lines

```
template<class P> vector<P> segInter(P a, P b, P c, P d){
    auto oa = c.cross(d, a), ob = c.cross(d, b),
        oc = a.cross(b, c), od = a.cross(b, d);
    // Checks if intersection is single non-endpoint point.
    if(sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> s;
    if(onSegment(c, d, a)) s.insert(a);
    if(onSegment(c, d, b)) s.insert(b);
    if(onSegment(a, b, c)) s.insert(c);
    if(onSegment(a, b, d)) s.insert(d);
    return {all(s)};
}
```

lineIntersection.h

Description:

If a unique intersection point of the lines going through s1, e1 and s2, e2 exists {1, point} is returned. If no intersection point exists {0, (0, 0)} is returned and if infinitely many exists {-1, (0, 0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



8.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.

Usage: vector<P> v = {P{4, 4}, P{1, 2}, P{2, 1}};

bool in = inPolygon(v, P{3, 3}, false);

Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h"	2bf504, 11 lines
<pre>template<class P> bool inPolygon(vector<P> &p, P a, bool strict = true){ int cnt = 0, n = sz(p); rep(i, 0, n){ P q = p[(i + 1) % n]; if(onSegment(p[i], q, a)) return !strict; //or: if(segDist(p[i], q, a) <= eps) return !strict; cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0; } return cnt; }</pre>	

PolygonArea.h

Description: Returns twice the signed area of a polygon. Clockwise enumeration gives negative area. Watch out for overflow if using int as T!

"Point.h"	f12300, 6 lines
<pre>template<class T> T polygonArea2(vector<Point<T>>& v){ T a = v.back().cross(v[0]); rep(i, 0, sz(v)-1) a += v[i].cross(v[i+1]); return a; }</pre>	

PolygonCenter.h

Description: Returns the center of mass for a polygon.

Time: $\mathcal{O}(n)$

"Point.h"	9706dc, 9 lines
<pre>typedef Point<double> P; P polygonCenter(const vector<P>& v){ P res(0, 0); double A = 0; for(int i = 0, j = sz(v) - 1; i < sz(v); j = i++){ res = res + (v[i] + v[j]) * v[j].cross(v[i]); A += v[j].cross(v[i]); } return res / A / 3; }</pre>	

PolygonCut.h

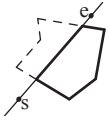
Description:

Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.

Usage: vector<P> p = ...;

p = polygonCut(p, P(0, 0), P(1, 0));

"Point.h"	d07181, 13 lines
<pre>typedef Point<double> P; vector<P> polygonCut(const vector<P>& poly, P s, P e){ vector<P> res; rep(i, 0, sz(poly)){ P cur = poly[i], prev = i ? poly[i-1] : poly.back(); auto a = s.cross(e, cur), b = s.cross(e, prev); if((a < 0) != (b < 0)) res.push_back(cur + (prev - cur) * (a / (a - b))); if(a < 0) res.push_back(cur); } return res; }</pre>	



ConvexHull.h

Description:

Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull.

Time: $\mathcal{O}(n \log n)$

"Point.h"	310954, 13 lines
<pre>typedef Point<ll> P; vector<P> convexHull(vector<P> pts){ if(sz(pts) <= 1) return pts; sort(all(pts)); vector<P> h(sz(pts)+1); int s = 0, t = 0; for(int it = 2; it--; s = --t, reverse(all(pts))) for(P p : pts){ while(t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--; h[t++] = p; } return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])}; }</pre>	



HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).

Time: $\mathcal{O}(n)$

"Point.h"	c571b8, 12 lines
<pre>typedef Point<ll> P; array<P, 2> hullDiameter(vector<P> S){ int n = sz(S), j = n < 2 ? 0 : 1; pair<ll, array<P, 2>> res({0, {S[0], S[0]}}); rep(i, 0, j) for(;; j = (j + 1) % n){ res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}}); if((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0) break; } return res.second; }</pre>	

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.

Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h"	71446b, 14 lines
<pre>typedef Point<ll> P; bool inHull(const vector<P>& l, P p, bool strict = true){ int a = 1, b = sz(l) - 1, r = !strict; if(sz(l) < 3) return r && onSegment(l[0], l.back(), p); if(sideOf(l[0], l[a], l[b]) > 0) swap(a, b); if(sideOf(l[0], l[a], p) >= r sideOf(l[0], l[b], p) <= -r) return false; while(abs(a - b) > 1){ int c = (a + b) / 2; (sideOf(l[0], l[c], p) > 0 ? b : a) = c; } return sgn(l[a].cross(l[b], p)) < r; }</pre>	

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet(-1, -1)$ if no collision, $\bullet(i, -1)$ if touching the corner i , $\bullet(i, i)$ if along side $(i, i + 1)$, $\bullet(i, j)$ if crossing sides $(i, i + 1)$ and $(j, j + 1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i, i + 1)$. The points are returned in the same order as the line hits the polygon. extrVertex returns the point of a hull with the max projection onto a line.

Time: $\mathcal{O}(\log n)$

"Point.h"	7cf45b, 39 lines
<pre>#define cmp(i, j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n])) #define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0 template <class P> int extrVertex(vector<P>& poly, P dir){ int n = sz(poly), lo = 0, hi = n; if(extr(0)) return 0; while(lo + 1 < hi){ int m = (lo + hi) / 2; if(extr(m)) return m; int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m); (ls < ms (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m; } return lo; } #define cmpL(i) sgn(a.cross(poly[i], b)) template <class P> array<int, 2> lineHull(P a, P b, vector<P>& poly){ int endA = extrVertex(poly, (a - b).perp()); int endB = extrVertex(poly, (b - a).perp()); if(cmpL(endA) < 0 cmpL(endB) > 0) return {-1, -1}; array<int, 2> res; rep(i, 0, 2){ int lo = endB, hi = endA, n = sz(poly); while((lo + 1) % n != hi){ int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n; (cmpL(m) == cmpL(endB) ? lo : hi) = m; } res[i] = (lo + !cmpL(hi)) % n; swap(endA, endB); } if(res[0] == res[1]) return {res[0], -1}; if(!cmpL(res[0]) && !cmpL(res[1])) switch((res[0] - res[1] + sz(poly) + 1) % sz(poly)){ case 0: return {res[0], res[0]}; case 2: return {res[1], res[1]}; } return res; }</pre>	

8.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.

Time: $\mathcal{O}(n \log n)$

"Point.h"	ac41a6, 17 lines
<pre>typedef Point<ll> P; pair<P, P> closest(vector<P> v){ assert(sz(v) > 1); set<P> S; sort(all(v), [](P a, P b){ return a.y < b.y; }); pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}}; int j = 0; for(P p : v){ P d(1 + (ll)sqrt(ret.first), 0); while(v[j].y <= p.y - d.x) S.erase(v[j++]); auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d); for(; lo != hi; ++lo)</pre>	

```
        ret = min(ret, {( *lo - p).dist2(), { *lo, p}}});
        S.insert(p);
    }
    return ret.second;
}
```

kdTree.h

Description: KD-tree (2d, can be extended to 3d)

"Point.h"69ff55, 63 lines

```
typedef long long T;
typedef Point<T> P;
const T INF = numeric_limits<T>::max();
```

```
bool on_x(P a, P b){ return a.x < b.x; }
bool on_y(P a, P b){ return a.y < b.y; }
```

```
struct Node {
    P pt; // if this is a leaf, the single point in it
    T x0 = INF, x1 = -INF, y0 = INF, y1 = -INF; // bounds
    Node *first = 0, *second = 0;

    T distance(P p){ // min squared distance to a point
        T x = (p.x < x0 ? x0 : p.x > x1 ? x1 : p.x);
        T y = (p.y < y0 ? y0 : p.y > y1 ? y1 : p.y);
        return (P(x, y) - p).dist2();
    }

    Node(vector<P>&& vp) : pt(vp[0]){
        for(P p : vp){
            x0 = min(x0, p.x); x1 = max(x1, p.x);
            y0 = min(y0, p.y); y1 = max(y1, p.y);
        }
        if(vp.size() > 1){
            // split on x if width >= height (not ideal...)
            sort(all(vp), x1 - x0 >= y1 - y0 ? on_x : on_y);
            // divide by taking half the array for each child (not
            // best performance with many duplicates in the middle)
            int half = sz(vp)/2;
            first = new Node({vp.begin(), vp.begin() + half});
            second = new Node({vp.begin() + half, vp.end()});
        }
    }
};
```

```
struct KDTree {
    Node* root;
    KDTree(const vector<P>& vp) : root(new Node({all(vp)})){}

    pair<T, P> search(Node *node, P p){
        if(!node->first){
            // uncomment if we should not find the point itself:
            // if(p == node->pt) return {INF, P()};
            return make_pair((p - node->pt).dist2(), node->pt);
        }

        Node *f = node->first, *s = node->second;
        T bfirst = f->distance(p), bsec = s->distance(p);
        if(bfirst > bsec) swap(bsec, bfirst), swap(f, s);

        // search closest side first, other side if needed
        auto best = search(f, p);
        if(bsec < best.first)
            best = min(best, search(s, p));
        return best;
    }

    // find nearest point to a point, and its squared distance
    // (requires an arbitrary operator< for Point)
    pair<T, P> nearest(P p){
```

kdTree FastDelaunay PolyhedronVolume Point3D

```
        return search(root, p);
    }
};

FastDelaunay.h
Description: Fast Delaunay triangulation. Each circumcircle contains none
of the input points. There must be no duplicate points. If all points are on a
line, no triangles will be returned. Should work for doubles as well, though
there may be precision issues in 'circ'. Returns triangles in order {t[0][0],
t[0][1], t[0][2], t[1][0], ...}, all counter-clockwise.
Time: O(n log n)
"Point.h"99ac27, 88 lines

typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t ll1; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX, LLONG_MAX); // not equal to any other point

struct Quad {
    Q rot=0, o=0; P p=arb; bool mark=false;
    P& F(){ return r()->p; }
    Q& r(){ return rot->rot; }
    Q prev(){ return rot->o->rot; }
    Q next(){ return r()->prev(); }
} *H;

bool circ(P p, P a, P b, P c){ // is p in the circumcircle?
    ll1 p2 = p.dist2(), A = a.dist2()-p2,
        B = b.dist2()-p2, C = c.dist2()-p2;
    return p.cross(a, b)*C + p.cross(b, c)*A + p.cross(c, a)*B >
        0;
}

Q makeEdge(P orig, P dest){
    Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
    H = r->o; r->r()->r() = r;
    rep(i, 0, 4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r
        ();
    r->p = orig; r->F() = dest;
    return r;
}

void splice(Q a, Q b){
    swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
}

Q connect(Q a, Q b){
    Q q = makeEdge(a->F(), b->p);
    splice(q, a->next());
    splice(q->r(), b);
    return q;
}

pair<Q, Q> rec(const vector<P>&& s){
    if(sz(s) <= 3){
        Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
        if(sz(s) == 2) return {a, a->r()};
        splice(a->r(), b);
        auto side = s[0].cross(s[1], s[2]);
        Q c = side ? connect(b, a) : 0;
        return {side < 0 ? c->r() : a, side < 0 ? c : b->r()};
    }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
    Q A, B, ra, rb;
    int half = sz(s) / 2;
    tie(ra, A) = rec({all(s) - half});
    tie(B, rb) = rec({sz(s) - half + all(s)});
    while((B->p.cross(H(A)) < 0 && (A = A->next())) ||
        (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
    Q base = connect(B->r(), A);
    if(A->p == ra->p) ra = base->r();
```

```
    if(B->p == rb->p) rb = base;

#define DEL(e, init, dir) Q e = init->dir; if(valid(e)) \
    while(circ(e->dir->F(), H(base), e->F())){ \
        Q t = e->dir; \
        splice(e, e->prev()); \
        splice(e->r(), e->r()->prev()); \
        e->o = H; H = e; e = t; \
    }
    for(;;){
        DEL(LC, base->r(), o); DEL(RC, base, prev());
        if(!valid(LC) && !valid(RC)) break;
        if(!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
            base = connect(RC, base->r());
        else
            base = connect(base->r(), LC->r());
    }
    return { ra, rb };
}
```

```
vector<P> triangulate(vector<P> pts){
    sort(all(pts)); assert(unique(all(pts)) == pts.end());
    if(sz(pts) < 2) return {};
    Q e = rec(pts).first;
    vector<Q> q = {e};
    int qi = 0;
    while(e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
    q.push_back(c->r()); c = c->next(); }while(c != e); }
    ADD; pts.clear();
    while(qi < sz(q)) if(!(e = q[qi++])->mark) ADD;
    return pts;
}
```

8.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6 lines

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilst){
    double v = 0;
    for(auto i : trilst) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
    return v / 6;
}
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

166780, 38 lines

```
template<class T> struct Point3D {
    typedef Point3D P;
    T x, y, z;
    explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z){}
    bool operator<(P p) const {
        return tie(x, y, z) < tie(p.x, p.y, p.z); }
    bool operator==(P p) const {
        return tie(x, y, z) == tie(p.x, p.y, p.z); }
    P operator+(P p) const { return P(x+p.x, y+p.y, z+p.z); }
    P operator-(P p) const { return P(x-p.x, y-p.y, z-p.z); }
    P operator*(T d) const { return P(x*d, y*d, z*d); }
    P operator/(T d) const { return P(x/d, y/d, z/d); }
    T dot(P p) const { return x*p.x + y*p.y + z*p.z; }
    P cross(P p) const {
        return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
    }
    T dist2() const { return x*x + y*y + z*z; }
    double dist() const { return sqrt((double)dist2()); }
    //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
```

```
double phi() const { return atan2(y, x); }
//Zenith angle (latitude) to the z-axis in interval [0, pi]
double theta() const { return atan2(sqrt(x*x+y*y), z); }
P unit() const { return *this/(T)dist(); } //makes dist()==1
//returns unit vector normal to *this and p
P normal(P p) const { return cross(p).unit(); }
//returns point rotated 'angle' radians ccw around axis
P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
}
};

pair<double, double> planeProj(Point3D<double> a, Point3D<double> b,
    Point3D<double> c, Point3D<double> p){
    auto x = (b - a).unit(), y = c - a;
    y = (y - x * y.dot(x)).unit(); p = p - a;
    return {p.dot(x), p.dot(y)};
}
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.
Time: $\mathcal{O}(n^2)$

"Point3D.h"	5b45fc, 49 lines
-------------	------------------

typedef Point3D<double> P3;

```
struct PR {
    void ins(int x){ (a == -1 ? a : b) = x; }
    void rem(int x){ (a == x ? a : b) = -1; }
    int cnt(){ return (a != -1) + (b != -1); }
    int a, b;
};
```

struct F { P3 q; int a, b, c; };

```
vector<F> hull3d(const vector<P3>& A){
    assert(sz(A) >= 4);
    vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x, y) E[f.x][f.y]
    vector<F> FS;
    auto mf = [&](int i, int j, int k, int l){
        P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
        if(q.dot(A[l]) > q.dot(A[i]))
            q = q * -1;
        F f{q, i, j, k};
        E(a, b).ins(k); E(a, c).ins(j); E(b, c).ins(i);
        FS.push_back(f);
    };
    rep(i, 0, 4) rep(j, i+1, 4) rep(k, j+1, 4)
        mf(i, j, k, 6 - i - j - k);
```

```
rep(i, 4, sz(A)){
    rep(j, 0, sz(FS)){
        F f = FS[j];
        if(f.q.dot(A[i]) > f.q.dot(A[f.a])){
            E(a, b).rem(f.c);
            E(a, c).rem(f.b);
            E(b, c).rem(f.a);
            swap(FS[j--], FS.back());
            FS.pop_back();
        }
    }
    int nw = sz(FS);
    rep(j, 0, nw){
        F f = FS[j];
#define C(a, b, c) if(E(a, b).cnt() != 2) mf(f.a, f.b, i, f.c);
```

```
        C(a, b, c); C(a, c, b); C(b, c, a);
    }
}
for(F& it : FS) if((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
return FS;
};
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (latitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius){
    double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
    double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
    double dz = cos(t2) - cos(t1);
    double d = sqrt(dx*dx + dy*dy + dz*dz);
    return radius*2*asin(min(1.0, d/2));
}
```

Strings (9)

KMP.h

Description: pi[x] computes the length of the longest prefix of s that ends at x, other than s[0..x] itself (abacaba -> 0010123). Can be used to find all occurrences of a string.
Time: $\mathcal{O}(n)$

	d4375c, 16 lines
--	------------------

```
vi pi(const string& s){
    vi p(sz(s));
    rep(i, 1, sz(s)){
        int g = p[i-1];
        while(g && s[i] != s[g]) g = p[g-1];
        p[i] = g + (s[i] == s[g]);
    }
    return p;
}
```

```
vi match(const string& s, const string& pat){
    vi p = pi(pat + '\0' + s), res;
    rep(i, sz(p)-sz(s), sz(p))
        if(p[i] == sz(pat)) res.push_back(i - 2 * sz(pat));
    return res;
}
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

	ee09e2, 12 lines
--	------------------

```
vi Z(const string& S){
    vi z(sz(S));
    int l = -1, r = -1;
    rep(i, 1, sz(S)){
        z[i] = i >= r ? 0 : min(r - i, z[i - l]);
        while(i + z[i] < sz(S) && S[i + z[i]] == S[z[i]])
            z[i]++;
        if(i + z[i] > r)
            l = i, r = i + z[i];
    }
    return z;
}
```

}

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

	e7ad79, 13 lines
--	------------------

```
array<vi, 2> manacher(const string& s){
    int n = sz(s);
    array<vi, 2> p = {vi(n+1), vi(n)};
    rep(z, 0, 2) for(int i=0, l=0, r=0; i < n; i++){
        int t = r-i+!z;
        if(i<r) p[z][i] = min(t, p[z][l+t]);
        int L = i-p[z][i], R = i+p[z][i]-!z;
        while(L>=1 && R+1<n && s[L-1] == s[R+1])
            p[z][i]++, L--, R++;
        if(R>r) l=L, r=R;
    }
    return p;
}
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: rotate(v.begin(), v.begin()+minRotation(v), v.end());
Time: $\mathcal{O}(N)$

	67652e, 8 lines
--	-----------------

```
int minRotation(string s){
    int a=0, N=sz(s); s += s;
    rep(b, 0, N) rep(k, 0, N){
        if(a+k == b || s[a+k] < s[b+k]){b += max<int>(0, k-1);
            break;}
        if(s[a+k] > s[b+k]){a = b; break;}
    }
    return a;
}
```

SuffixArray.h

Description: Builds suffix array for a string. sa[i] is the starting index of the suffix which is i'th in the sorted suffix array. The returned vector is of size n + 1, and sa[0] = n. The lcp array contains longest common prefixes for neighbouring strings in the suffix array: lcp[i] = lcp(sa[i], sa[i-1]), lcp[0] = 0. The input string must not contain any nul chars.
Time: $\mathcal{O}(n \log n)$

	ed2029, 26 lines
--	------------------

```
struct SuffixArray {
    vi sa, lcp;
    SuffixArray(string s, int lim=256){ // or vector<int>
        s.push_back(0); int n = sz(s), k = 0, a, b;
        vi x(n), y(n), ws(max(n, lim));
        rep(i, 0, n) x[i] = (unsigned char)s[i];
        sa = lcp = y, iota(all(sa), 0);
        for(int j = 0, p = 0; p < n; j = max<int>(1, j * 2), lim = p){
            p = j, iota(all(y), n - j);
            rep(i, 0, n) if(sa[i] >= j) y[p++] = sa[i] - j;
            fill(all(ws), 0);
            rep(i, 0, n) ws[x[i]]++;
            rep(i, 1, lim) ws[i] += ws[i - 1];
            for(int i = n; i--;) sa[--ws[x[i]]] = y[i];
            swap(x, y), p = 1, x[sa[0]] = 0;
            rep(i, 1, n) a = sa[i - 1], b = sa[i], x[b] =
                (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p++;
        }
        for(int i = 0, j; i < n - 1; i++){
            if(k) k--;
            j = sa[x[i] - 1];
            while(s[i + k] == s[j + k]) k++;
            lcp[x[i]] = k;
        }
    }
};
```

```
    }
  }
};
```

SuffixArraySAIS.h

Description: Linear suffix array by SA-IS. Includes the empty suffix.

Time: $\mathcal{O}(n)$

e66dbd, 66 lines

```
struct SAIS {
    vi sa, lcp;
    vi build(const vi &s, int lim){
        int n = sz(s);
        if(n == 1) return {0};
        vi ret(n, -1), ls(n), sumL(lim + 1), sumS(lim + 1);
        for(int i = n - 2; i >= 0; i--){
            ls[i] = s[i] == s[i + 1] ? ls[i + 1] : s[i] < s[i + 1];
        }
        rep(i, 0, n) (ls[i] ? sumL[s[i] + 1] : sumS[s[i]])++;
        rep(i, 0, lim + 1){
            sumS[i] += sumL[i];
            if(i < lim) sumL[i + 1] += sumS[i];
        }
        auto induce = [&](const vi &lms){
            fill(all(ret), -1);
            vi buf = sumS;
            for(int x : lms) ret[buf[s[x]]++] = x;
            buf = sumL, ret[buf[s[n - 1]]++] = n - 1;
            rep(i, 0, n){
                int x = ret[i] - 1;
                if(x >= 0 && !ls[x]) ret[buf[s[x]]++] = x;
            }
            buf = sumL;
            for(int i = n; i--;){
                int x = ret[i] - 1;
                if(x >= 0 && ls[x]) ret[--buf[s[x] + 1]] = x;
            }
        };
        vi id(n + 1, -1), lms;
        rep(i, 1, n) if(!ls[i - 1] && ls[i]) id[i] = sz(lms), lms.pb(i);
        induce(lms);
        if(sz(lms)){
            vi ord;
            for(int x : ret) if(id[x] != -1) ord.pb(x);
            vi rec(sz(lms)); int names = 0;
            rec[id[ord[0]]] = 0;
            rep(i, 1, sz(lms)){
                int l = ord[i - 1], r = ord[i];
                int el = id[l] + 1 < sz(lms) ? lms[id[l] + 1] : n;
                int er = id[r] + 1 < sz(lms) ? lms[id[r] + 1] : n;
                bool same = el - 1 == er - r;
                while(same && l < el && s[l] == s[r]) l++, r++;
                if(l == n || s[l] != s[r]) same = false;
                rec[id[ord[i]]] = same ? names : ++names;
            }
            vi recSa = build(rec, names);
            rep(i, 0, sz(lms)) ord[i] = lms[recSa[i]];
            induce(ord);
        }
        return ret;
    }
};
SAIS(string s){
    int n = sz(s), k = 0;
    vi a(n + 1), rk(n + 1);
    rep(i, 0, n) a[i] = (unsigned char)s[i];
    sa = build(a, 255), lcp.assign(n + 1, 0);
    rep(i, 0, n + 1) rk[sa[i]] = i;
    rep(i, 0, n){
        if(k) k--;
```

SuffixArraySAIS SuffixArrayDC3 SuffixTree

```
        int j = sa[rk[i] - 1];
        while(a[i + k] == a[j + k]) k++;
        lcp[rk[i]] = k;
    }
};
```

SuffixArrayDC3.h

Description: Linear suffix array by DC3/skew. Includes the empty suffix.

Time: $\mathcal{O}(n)$

2d1263, 79 lines

```
struct DC3 {
    vi sa, lcp;
    bool leq(int a, int b, int c, int d){
        return a < c || (a == c && b <= d);
    }
    bool leq(int a, int b, int c, int d, int e, int f){
        return a < d || (a == d && leq(b, c, e, f));
    }
    void radix(const vi &a, vi &b, const vi &r, int add, int n,
        int lim){
        vi cnt(lim + 1);
        rep(i, 0, n) cnt[r[a[i] + add]]++;
        for(int i = 0, sum = 0; i <= lim; i++){
            int x = cnt[i]; cnt[i] = sum, sum += x;
        }
        rep(i, 0, n) b[cnt[r[a[i] + add]]++] = a[i];
    }
};
vi build(vi &s, int n, int lim){
    if(n == 0) return {};
    if(n == 1) return {0};
    int n0 = (n + 2) / 3, n1 = (n + 1) / 3;
    int n2 = n / 3, n02 = n0 + n2;
    vi s12(n02 + 3), sa12(n02 + 3), s0(n0), sa0(n0);
    for(int i = 0, j = 0; i < n + n0 - n1; i++){
        if(i % 3) s12[j++] = i;
    }
    radix(s12, sa12, s, 2, n02, lim);
    radix(sa12, s12, s, 1, n02, lim);
    radix(s12, sa12, s, 0, n02, lim);
    int names = 0, a = -1, b = -1, c = -1;
    rep(i, 0, n02){
        int x = sa12[i];
        if(s[x] != a || s[x + 1] != b || s[x + 2] != c){
            names++, a = s[x], b = s[x + 1], c = s[x + 2];
        }
        s12[x / 3 + (x % 3 == 2) * n0] = names;
    }
    if(names < n02){
        vi rec = build(s12, n02, names);
        rep(i, 0, n02) sa12[i] = rec[i], s12[rec[i]] = i + 1;
    }
    else rep(i, 0, n02) sa12[s12[i] - 1] = i;
    for(int i = 0, j = 0; i < n02; i++){
        if(sa12[i] < n0) s0[j++] = 3 * sa12[i];
    }
    radix(s0, sa0, s, 0, n0, lim);
    vi ans; ans.reserve(n);
    int p = 0, t = n0 - n1;
    auto pos = [&](int x){
        return x < n0 ? 3 * x + 1 : 3 * (x - n0) + 2;
    };
    while(p < n0 && t < n02){
        int x = pos(sa12[t]), y = sa0[p];
        bool take = sa12[t] < n0 ?
            leq(s[x], s12[sa12[t] + n0], s[y], s12[y / 3]) :
            leq(s[x], s[x + 1], s12[sa12[t] - n0 + 1],
                s[y], s[y + 1], s12[y / 3 + n0]);
        ans.pb(take ? x : y);
```

```
        if(take) t++; else p++;
    }
    while(p < n0) ans.pb(sa0[p++]);
    while(t < n02) ans.pb(pos(sa12[t++]));
    return ans;
}
DC3(string s){
    int n = sz(s), k = 0;
    vi a(n + 3), rk(n + 1);
    rep(i, 0, n) a[i] = (unsigned char)s[i];
    vi raw = build(a, n, 255);
    sa.pb(n); sa.insert(sa.end(), all(raw));
    lcp.assign(n + 1, 0);
    rep(i, 0, n + 1) rk[sa[i]] = i;
    rep(i, 0, n){
        if(k) k--;
        int j = sa[rk[i] - 1];
        while(i + k < n && j + k < n && a[i + k] == a[j + k]) k
            ++;
        lcp[rk[i]] = k;
    }
};
```

SuffixTree.h

Description: Ukkonen suffix tree for lowercase strings. Nodes store [l, r).

Time: $\mathcal{O}(28N)$

a14fe6, 57 lines

```
struct SuffixTree {
    enum { ALPHA = 28 };
    int toi(char c){ return c - 'a'; }
    string a;
    vector<array<int, ALPHA>> t;
    vi l, r, p, s;
    int v = 0, q = 0, m = 2;
    void ukkadd(int i, int c){ suff:
        if(r[v] <= q){
            if(t[v][c] == -1){
                t[v][c] = m; l[m] = i; p[m++] = v; v = s[v]; q = r[v];
                goto suff;
            }
            v = t[v][c]; q = l[v];
        }
        if(q == -1 || c == toi(a[q])) q++;
        else{
            l[m+1] = i; p[m+1] = m; l[m] = l[v]; r[m] = q;
            p[m] = p[v]; t[m][c] = m+1; t[m][toi(a[q])] = v;
            l[v] = q; p[v] = m; t[p[m]][toi(a[l[m]])] = m;
            v = s[p[m]]; q = l[m];
            while(q < r[m]) v = t[v][toi(a[q])], q += r[v] - l[v];
            s[m] = q == r[m] ? v : m+2;
            q = r[v] - (q - r[m]); m += 2;
            goto suff;
        }
    }
    SuffixTree(string a) : a(a), t(2 * sz(a) + 5), l(t.size()), r(t.size(), sz(a)),
        p(t.size()), s(t.size()){
        for(auto &x : t) x.fill(-1);
        t[1].fill(0); s[0] = 1;
        l[0] = l[1] = -1; r[0] = r[1] = p[0] = p[1] = 0;
        rep(i, 0, sz(a)) ukkadd(i, toi(a[i]));
    }
    ll distinctSubstrings(){
        ll ans = 0;
        rep(i, 2, m) ans += r[i] - l[i];
        return ans;
    }
    pii best;
```

24

```
int lcs(int node, int i1, int i2, int len){
    if(l[node] <= i1 && i1 < r[node]) return 1;
    if(l[node] <= i2 && i2 < r[node]) return 2;
    int mask = 0;
    if(node) len += r[node]-l[node];
    rep(c, 0, ALPHA) if(t[node][c] != -1){
        mask |= lcs(t[node][c], i1, i2, len);
    }
    if(mask == 3) best = max(best, {len, r[node]-len});
    return mask;
}
static pii LCS(string s, string t){
    SuffixTree st(s+' '+t+'|');
    st.lcs(0, sz(s), sz(s)+1+sz(t), 0);
    return st.best;
}
};
```

AhoCorasick.h

Description: Aho-Corasick automaton, used for multiple pattern matching. Initialize with AhoCorasick ac(patterns); the automaton start node will be at index 0. find(word) returns for each position the index of the longest word that ends there, or -1 if none. findAll(—, word) finds all words (up to $N\sqrt{N}$ many if no duplicate patterns) that start at each position (shortest first). Duplicate patterns are allowed; empty patterns are not. To find the longest words that start at each position, reverse all input. For large alphabets, split each symbol into chunks, with sentinel bits for symbol boundaries. **Time:** construction takes $\mathcal{O}(26N)$, where N = sum of length of patterns. find(x) is $\mathcal{O}(N)$, where N = length of x. findAll is $\mathcal{O}(NM)$.

f35677, 66 lines

```
struct AhoCorasick {
    enum {alpha = 26, first = 'A'}; // change this!
    struct Node {
        // (nmatches is optional)
        int back, next[alpha], start = -1, end = -1, nmatches = 0;
        Node(int v){ memset(next, v, sizeof(next)); }
    };
    vector<Node> N;
    vi backp;
    void insert(string& s, int j){
        assert(!s.empty());
        int n = 0;
        for(char c : s){
            int& m = N[n].next[c - first];
            if(m == -1){ n = m = sz(N); N.emplace_back(-1); }
            else n = m;
        }
        if(N[n].end == -1) N[n].start = j;
        backp.push_back(N[n].end);
        N[n].end = j;
        N[n].nmatches++;
    }
    AhoCorasick(vector<string>& pat) : N(1, -1){
        rep(i, 0, sz(pat)) insert(pat[i], i);
        N[0].back = sz(N);
        N.emplace_back(0);

        queue<int> q;
        for(q.push(0); !q.empty(); q.pop()){
            int n = q.front(), prev = N[n].back;
            rep(i, 0, alpha){
                int &ed = N[n].next[i], y = N[prev].next[i];
                if(ed == -1) ed = y;
                else {
                    N[ed].back = y;
                    (N[ed].end == -1 ? N[ed].end : backp[N[ed].start])
                        = N[y].end;
                    N[ed].nmatches += N[y].nmatches;
                    q.push(ed);
                }
            }
        }
    }
};
```

```

    }
    }
}
vi find(string word){
    int n = 0;
    vi res; // ll count = 0;
    for(char c : word){
        n = N[n].next[c - first];
        res.push_back(N[n].end);
        // count += N[n].nmatches;
    }
    return res;
}
vector<vi> findAll(vector<string>& pat, string word){
    vi r = find(word);
    vector<vi> res(sz(word));
    rep(i, 0, sz(word)){
        int ind = r[i];
        while(ind != -1){
            res[i - sz(pat[ind]) + 1].push_back(ind);
            ind = backp[ind];
        }
    }
    return res;
}
};
```

Sam.h

Description: Generalized suffix automaton for lowercase strings. Construct with total length, insert strings, build, then solve for cnt.

Time: $\mathcal{O}(N\text{alphabet})$

5cc30c, 38 lines

```
struct exSAM{
    static const int C = 27;
    vector<int>len, link, ord, cnt;
    vector<array<int, C>>go;
    int last;
    exSAM(int n){
        len.reserve(4 * n + 5), link.reserve(4 * n + 5), cnt.
            reserve(4 * n + 5), go.reserve(4 * n + 5);
        node(); link[0] = -1, last = 0;
    }
    int node(){
        len.pb(0), link.pb(0), cnt.pb(0), go.pb({}); return sz(len)
            - 1;
    }
    void extend(int c){
        int cur = node(), p = last; len[cur] = len[last] + 1, cnt[
            cur] = 1, last = cur;
        for(; p != -1 and !go[p][c]; p = link[p]) go[p][c] = cur;
        if(p == -1) return link[cur] = 0, void();
        int q = go[p][c];
        if(len[p] + 1 == len[q]) return link[cur] = q, void();
        int cl = node(); go[cl] = go[q], len[cl] = len[p] + 1, link
            [cl] = link[q];
        for(; p != -1 and go[p][c] == q; p = link[p]) go[p][c] = cl
            ;
        link[q] = link[cur] = cl;
    }
    void insert(const string &s){
        for(char c : s) extend(c - 'a');
        extend(26);
    }
    void build(){
        int n = sz(len); ord.resize(n); vector<int>sum(len[last] +
            1);
        rep(i, 0, n) ++sum[len[i]];
        partial_sum(all(sum), sum.begin());
    }
};
```

```
        rep(i, 0, n) ord[--sum[len[i]]] = i;
    }
    void solve(){
        for(int i = sz(ord) - 1; i; --i){
            int u = ord[i]; cnt[link[u]] += cnt[u];
        }
    }
};
```

Various (10)

10.1 Intervals

IntervalContainer.h

Description: Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).

Time: $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R){
    if(L == R) return is.end();
    auto it = is.lower_bound({L, R}), before = it;
    while(it != is.end() && it->first <= R){
        R = max(R, it->second);
        before = it = is.erase(it);
    }
    if(it != is.begin() && (--it)->second >= L){
        L = min(L, it->first);
        R = max(R, it->second);
        is.erase(it);
    }
    return is.insert(before, {L, R});
}

void removeInterval(set<pii>& is, int L, int R){
    if(L == R) return;
    auto it = addInterval(is, L, R);
    auto r2 = it->second;
    if(it->first == L) is.erase(it);
    else (int&)it->second = L;
    if(R != r2) is.emplace(R, r2);
}
```

IntervalCover.h

Description: Compute indices of smallest set of intervals covering another interval. Intervals should be [inclusive, exclusive). To support [inclusive, inclusive], change (A) to add || R.empty(). Returns empty set on failure (or if G is empty).

Time: $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T, T> G, vector<pair<T, T>> I){
    vi S(sz(I)), R;
    iota(all(S), 0);
    sort(all(S), [&](int a, int b){ return I[a] < I[b]; });
    T cur = G.first;
    int at = 0;
    while(cur < G.second){ // (A)
        pair<T, int> mx = make_pair(cur, -1);
        while(at < sz(I) && I[S[at]].first <= cur){
            mx = max(mx, make_pair(I[S[at]].second, S[at]));
            at++;
        }
        if(mx.second == -1) return {};
        cur = mx.first;
        R.push_back(mx.second);
    }
    return R;
}
```

```
}

```

ConstantIntervals.h

Description: Split a monotone function on [from, to) into a minimal set of half-open intervals on which it has the same value. Runs a callback g for each such interval.

Usage: constantIntervals(0, sz(v), [&](int x){return v[x];},
[&](int lo, int hi, T val){...});
Time: $\mathcal{O}\left(k\log\frac{n}{k}\right)$

	753a4c, 19 lines
<pre>template<class F, class G, class T> void rec(int from, int to, F& f, G& g, int& i, T& p, T q){ if(p == q) return; if(from == to){ g(i, to, p); i = to; p = q; }else{ int mid = (from + to) >> 1; rec(from, mid, f, g, i, p, f(mid)); rec(mid+1, to, f, g, i, p, q); } } template<class F, class G> void constantIntervals(int from, int to, F f, G g){ if(to <= from) return; int i = from; auto p = f(i), q = f(to-1); rec(from, to-1, f, g, i, p, q); g(i, to, q); }</pre>	

10.2 Misc. algorithms

TernarySearch.h

Description: Find the smallest i in $[a,b]$ that maximizes $f(i)$, assuming that $f(a) < \dots < f(i) \geq \dots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $\leq$, and reverse the loop at (B). To minimize f , change it to $>$, also at (B).

Usage: int ind = ternSearch(0, n-1, [&](int i){return a[i];});
Time: $\mathcal{O}(\log(b-a))$

	9155b4, 11 lines
<pre>template<class F> int ternSearch(int a, int b, F f){ assert(a <= b); while(b - a >= 5){ int mid = (a + b) / 2; if(f(mid) < f(mid+1)) a = mid; // (A) else b = mid+1; } rep(i, a+1, b+1) if(f(a) < f(i)) a = i; // (B) return a; }</pre>	

LIS.h

Description: Compute indices for the longest increasing subsequence.

Time: $\mathcal{O}(N\log N)$

	2932a0, 17 lines
<pre>template<class I> vi lis(const vector<I>& S){ if(S.empty()) return {}; vi prev(sz(S)); typedef pair<I, int> p; vector<p> res; rep(i, 0, sz(S)){ // change 0 -> i for longest non-decreasing subsequence auto it = lower_bound(all(res), p{S[i], 0}); if(it == res.end()) res.emplace_back(), it = res.end()-1; *it = {S[i], i}; prev[i] = it == res.begin() ? 0 : (it-1)->second; } int L = sz(res), cur = res.back().second; vi ans(L);</pre>	

```
    while(L--) ans[L] = cur, cur = prev[cur];
    return ans;
}
```

FastKnapsack.h

Description: Given N non-negative integer weights w and a non-negative target t , computes the maximum $S \leq t$ such that S is the sum of some subset of the weights.

Time: $\mathcal{O}(N\max(w_i))$

	0c7a04, 16 lines
<pre>int knapsack(vi w, int t){ int a = 0, b = 0, x; while(b < sz(w) && a + w[b] <= t) a += w[b++]; if(b == sz(w)) return a; int m = *max_element(all(w)); vi u, v(2*m, -1); v[a+m-t] = b; rep(i, b, sz(w)){ u = v; rep(x, 0, m) v[x+w[i]] = max(v[x+w[i]], u[x]); for(x = 2*m; --x > m;) rep(j, max<int>(0, u[x]), v[x]) v[x-w[j]] = max(v[x-w[j]], j); } for(a = t; v[a+m-t] < 0; a--); return a; }</pre>	

10.3 Dynamic programming

KnuthDP.h

Description: When doing DP on intervals: $a[i][j] = \min_{i < k < j}(a[i][k] + a[k][j]) + f(i, j)$, where the (minimal) optimal k increases with both i and j , one can solve intervals in increasing order of length, and search $k = p[i][j]$ for $a[i][j]$ only between $p[i][j-1]$ and $p[i+1][j]$. This is known as Knuth DP. Sufficient criteria for this are if $f(b, c) \leq f(a, d)$ and $f(a, c) + f(b, d) \leq f(a, d) + f(b, c)$ for all $a \leq b \leq c \leq d$. Consider also: LineContainer (ch. Data structures), monotone queues, ternary search.

Time: $\mathcal{O}(N^2)$

DivideAndConquerDP.h

Description: $dp[i] = \min_{k \leq i} cost(i, k)$, assuming $opt[i]$ is monotone.

Time: $\mathcal{O}((N+K)\log N)$ cost calls

	149168, 18 lines
<pre>template<class F> pair<vector<ll>, vi> dcDP(int n, int k, F cost){ vector<ll>dp(n, LLONG_MAX); vi opt(n); auto go = [&](auto self, int l, int r, int ql, int qr)->void{ if(l >= r) return; int m = (l + r) / 2, hi = min(qr, m + 1), who = ql; rep(j, ql, hi){ ll v = cost(m, j); if(v < dp[m]) dp[m] = v, who = j; } opt[m] = who; self(self, l, m, ql, who + 1); self(self, m + 1, r, who, qr); }; go(go, 0, n, 0, k); return {dp, opt}; }</pre>	

10.4 Debugging tricks

- signal(SIGSEGV, [](int) { _Exit(0); });
converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). _GLIBCXX_DEBUG failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).

- feenableexcept(29); kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

10.5 Optimization tricks

10.5.1 Bit hacks

- $x \& -x$ is the least bit in x .
- for (int x = m; x;) { --x &= m; ... } loops over all subset masks of m (except m itself).
- $c = x\&-x, r = x+c; (((r^x) >> 2)/c) \mid r$ is the next number after x with the same number of bits set.
- rep(b,0,K) rep(i,0,(1 << K))
if (i & 1 << b) D[i] += D[i^(1 << b)];
computes all sums of subsets.

10.5.2 Pragmas

- #pragma GCC optimize ("Ofast") will make GCC auto-vectorize loops and optimizes floating points better.
- #pragma GCC optimize ("trapv") kills the program on integer overflows (but is really slow).

FastMod.h

Description: Compute $a\%b$ about 5 times faster than usual, where b is constant but not known at compile time. Returns a value congruent to $a \pmod b$ in the range $[0, 2b)$.

	751a02, 8 lines
<pre>typedef unsigned long long ull; struct FastMod { ull b, m; FastMod(ull b) : b(b), m(-1ULL / b){} ull reduce(ull a){ // a % b + (0 or b) return a - (ull)((__uint128_t(m) * a) >> 64) * b; } };</pre>	

FastInput.h

Description: Read an integer from stdin. Usage requires your program to pipe in input from file.

Usage: ./a.out < input.txt

Time: About 5x as fast as cin/scanf.

	7b3c70, 17 lines
<pre>inline char gc(){ // like getchar() static char buf[1 << 16]; static size_t bc, be; if(bc >= be){ buf[0] = 0, bc = 0; be = fread(buf, 1, sizeof(buf), stdin); } return buf[bc++]; // returns 0 on EOF }</pre>	

```
int readInt(){
    int a, c;
    while((a = gc()) < 40);
    if(a == '-') return -readInt();
    while((c = gc()) >= 48) a = a * 10 + c - 480;
    return a - 48;
}
```

BumpAllocator.h

Description: When you need to dynamically allocate many objects and don't care about freeing them. "new X" otherwise has an overhead of something like 0.05us + 16 bytes per allocation.

299041, 12 lines

```
// Either globally or in a single class:
alignas(max_align_t) static char buf[450 << 20];
static size_t bumpPos = sizeof buf;
void* operator new(size_t s){
    assert(s <= bumpPos);
    bumpPos = (bumpPos-s) & ~(alignof(max_align_t)-1);
    return (void*)&buf[bumpPos];
}
void operator delete(void*) noexcept {}
void operator delete(void*, size_t) noexcept {}
size_t bumpMark(){ return bumpPos; }
void bumpReset(size_t p){ assert(bumpPos <= p && p <= sizeof
    buf); bumpPos = p; }
```

SmallPtr.h

Description: A 32-bit pointer that points into BumpAllocator memory.

"BumpAllocator.h" 2dd6c9, 10 lines

```
template<class T> struct ptr {
    unsigned ind;
    ptr(T* p = 0) : ind(p ? unsigned((char*)p - buf) : 0){
        assert(ind < sizeof buf);
    }
    T& operator*() const { return *(T*)(buf + ind); }
    T* operator->() const { return &*this; }
    T& operator[](int a) const { return (&this)[a]; }
    explicit operator bool() const { return ind; }
};
```

BumpAllocatorSTL.h

Description: BumpAllocator for STL containers.
Usage: vector<vector<int, small<int>>> ed(N);

bb66d4, 14 lines

```
char buf[450 << 20] alignas(16);
size_t buf_ind = sizeof buf;

template<class T> struct small {
    typedef T value_type;
    small(){}
    template<class U> small(const U&){}
    T* allocate(size_t n){
        buf_ind -= n * sizeof(T);
        buf_ind &= 0 - alignof(T);
        return (T*)(buf + buf_ind);
    }
    void deallocate(T*, size_t){}
};
```

Unrolling.h

Description: Calls f(i) for i in [l, r), four iterations at a time.

a5aa6a, 5 lines

```
template<class F> void unroll4(int l, int r, F f){
    while((l&3) and l<r) f(l++);
    for(;l+4<=r;l+=4) f(l), f(l+1), f(l+2), f(l+3);
    while(l<r) f(l++);
}
```

Techniques (A)

techniques.txt	159 lines
Recursion	
Divide and conquer	
Finding interesting points in N log N	
Algorithm analysis	
Master theorem	
Amortized time complexity	
Greedy algorithm	
Scheduling	
Max contiguous subvector sum	
Invariants	
Huffman encoding	
Graph theory	
Dynamic graphs (extra book-keeping)	
Breadth first search	
Depth first search	
* Normal trees / DFS trees	
Dijkstra's algorithm	
MST: Prim's algorithm	
Bellman-Ford	
Konig's theorem and vertex cover	
Min-cost max flow	
Lovasz toggle	
Matrix tree theorem	
Maximal matching, general graphs	
Hopcroft-Karp	
Hall's marriage theorem	
Graphical sequences	
Floyd-Warshall	
Euler cycles	
Flow networks	
* Augmenting paths	
* Edmonds-Karp	
Bipartite matching	
Min. path cover	
Topological sorting	
Strongly connected components	
2-SAT	
Cut vertices, cut-edges and biconnected components	
Edge coloring	
* Trees	
Vertex coloring	
* Bipartite graphs (=> trees)	
* 3^n (special case of set cover)	
Diameter and centroid	
K'th shortest path	
Shortest cycle	
Dynamic programming	
Knapsack	
Coin change	
Longest common subsequence	
Longest increasing subsequence	
Number of paths in a dag	
Shortest path in a dag	
Dynprog over intervals	
Dynprog over subsets	
Dynprog over probabilities	
Dynprog over trees	
3^n set cover	
Divide and conquer	
Knuth optimization	
Convex hull optimizations	
RMQ (sparse table a.k.a 2^k-jumps)	
Bitonic cycle	
Log partitioning (loop over most restricted)	
Combinatorics	

Computation of binomial coefficients
Pigeon-hole principle
Inclusion/exclusion
Catalan number
Pick's theorem
Number theory
Integer parts
Divisibility
Euclidean algorithm
Modular arithmetic
* Modular multiplication
* Modular inverses
* Modular exponentiation by squaring
Chinese remainder theorem
Fermat's little theorem
Euler's theorem
Phi function
Frobenius number
Quadratic reciprocity
Pollard-Rho
Miller-Rabin
Hensel lifting
Vieta root jumping
Game theory
Combinatorial games
Game trees
Mini-max
Nim
Games on graphs
Games on graphs with loops
Grundy numbers
Bipartite games without repetition
General games without repetition
Alpha-beta pruning
Probability theory
Optimization
Binary search
Ternary search
Unimodality and convex functions
Binary search on derivative
Numerical methods
Numeric integration
Newton's method
Root-finding with binary/ternary search
Golden section search
Matrices
Gaussian elimination
Exponentiation by squaring
Sorting
Radix sort
Geometry
Coordinates and vectors
* Cross product
* Scalar product
Convex hull
Polygon cut
Closest pair
Coordinate-compression
Quadtrees
KD-trees
All segment-segment intersection
Sweeping
Discretization (convert to events and sweep)
Angle sweeping
Line sweeping
Discrete second derivatives
Strings
Longest common substring
Palindrome subsequences

Knuth-Morris-Pratt
Tries
Rolling polynomial hashes
Suffix array
Suffix tree
Aho-Corasick
Manacher's algorithm
Letter position lists
Combinatorial search
Meet in the middle
Brute-force with pruning
Best-first (A*)
Bidirectional search
Iterative deepening DFS / A*
Data structures
LCA (2^k-jumps in trees in general)
Pull/push-technique on trees
Heavy-light decomposition
Centroid decomposition
Lazy propagation
Self-balancing trees
Convex hull trick (wcipeg.com/wiki/Convex_hull_trick)
Monotone queues / monotone stacks / sliding queues
Sliding queue using 2 stacks
Persistent segment tree